Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow

Yixuan Mei
yixuanm@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Yonghao Zhuang
yzhuang2@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Xupeng Miao
xupeng@cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Juncheng Yang
juncheny@cs.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Zhihao Jia
zhihao@cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Rashmi Vinayak
rvinayak@cs.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Abstract

This paper introduces Helix, a distributed system for high-throughput, low-latency large language model (LLM) serving in heterogeneous GPU clusters. The key idea behind Helix is to formulate inference computation of LLMs over heterogeneous GPUs and network connections as a *max-flow* problem on directed, weighted graphs, whose nodes represent GPU instances and edges capture both GPU and network heterogeneity through their capacities. Helix then uses a mixed integer linear programming (MILP) algorithm to discover highly optimized strategies to serve LLMs on heterogeneous GPUs. This approach allows Helix to jointly optimize model placement and request scheduling, two highly entangled tasks in heterogeneous LLM serving. Our evaluation on several heterogeneous clusters ranging from 24 to 42 GPU nodes shows that Helix improves serving throughput by up to 3.3× and reduces prompting and decoding latency by up to 66% and 24%, respectively, compared to existing approaches. Helix is available at https://github.com/Thesys-lab/Helix-ASPLOS25.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Computing methodologies** → *Artificial intelligence*; *Parallel computing methodologies.*

Keywords: large language model serving, system for ML, distributed systems, cloud computing

ACM Reference Format:
Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3669940.3707215

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707215

Table 1. Minimum numbers of GPUs required to serve LLMs in existing homogeneous serving systems. We use half of GPU memory to store model parameters and the other half for key-value cache.

LLMs	Num. of Parameters	Num. of L4s	Num. of A100s	Num. of H100s
LLaMA-2 [56]	70 billion	12	7	4
GPT-3 [1]	175 billion	30	18	9
Grok-1 [59]	314 billion	53	32	16
LLaMA-3 [10]	405 billion	68	41	21

Table 2. Availability of different GPU instances in 6 regions on Google Compute Engine [11].

Region	GPU Type					
	H100	A100 80GB	A100 40GB	L4	T4	V100
us-central-1	✓	✓	✓	✓	✓	✓
us-east-4	✓	✓	×	✓	✓	×
us-east-1	×	×	✓	✓	✓	✓
eu-west-3	✓	×	×	✓	✓	×
asia-ne-1	✓	×	✓	✓	✓	×
asia-ne-3	×	×	✓	✓	✓	×

1 Introduction

Generative large language models (LLMs) such as GPT-4 [1] and LLaMA-3 [28] have demonstrated exceptional capabilities of creating natural language texts across a spectrum of application domains, including chatbot [38], coding assistant [26, 44], and task automation [15]. However, the increasingly large model sizes and high computational requirements of modern LLMs make it challenging to serve them cheaply and efficiently on modern cloud platforms. In particular, most of today's LLM serving systems (e.g., Orca [61] and vLLM [22]) target *homogeneous* GPU clusters [29], where all GPUs are of the same type and have identical memory capacity and compute resources. Due to increasing model

Helix：通过 Max-Flow 通过异构 GP Us 和网络为大型语言模型提供服务

梅以轩
yixuanm@andrew.cmu.edu
卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

庄永浩
yzhuang2@andrew.cmu.edu
卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

苗旭鹏
xupeng@cmu.edu 卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

杨俊成
juncheny@cs.cmu.edu 卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

贾志豪 zhihao@cmu.edu 卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

拉什米·维纳亚克
rvinayak@cs.cmu.edu 卡内基梅隆大学 美国宾夕法尼亚州匹兹堡

抽象的

本文介绍了 Helix，这是一个在异构 GPU 集群中提供高吞吐量、低延迟大型语言模型 (LLM) 的分布式系统。 Helix 背后的关键思想是将异构 GPU 和网络连接上的 LLM 推理计算公式化为有向加权图上的最大流问题，其节点代表 GPU 实例，边通过其能力捕获 GPU 和网络异构性。然后，Helix 使用混合整数线性规划 (MILP) 算法来发现高度优化的策略，为异构 GPU 上的大语言模型服务。这种方法允许 Helix 联合优化模型放置和请求调度，这是异构 LLM 服务中两个高度纠缠的任务。我们对 24 到 42 个 GPU 节点的多个异构集群的评估表明，与现有方法相比，Helix 将服务吞吐量提高了 3.3×，并将提示和解码延迟分别减少了 66% 和 24%。 Helix 可从 https://github.com/Thesys-lab/Helix-ASPLOS25 获取。

CCS 概念： • 计算机系统组织 → 云计算； • 计算方法 → 人工智能；并行计算方法。

关键词：大语言模型服务、机器学习系统、分布式系统、云计算

ACM参考格式：
梅宜轩、庄永浩、缪旭鹏、杨俊成、贾志浩和 Rashmi Vinayak。 2025. Helix：通过 Max-Flow 在异构 GPU 和网络上为大型语言模型提供服务。在第 30 届 ACM 国际编程语言和操作系统架构支持会议的会议记录中，

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707215

表 1. 在现有同类服务系统中为 LLM 提供服务所需的最低 GPU 数量。我们使用一半的 GPU 内存来存储模型参数，另一半用于键值缓存。

LLMs	Num. of Parameters	Num. of L4s	Num. of A100s	Num. of H100s
LLaMA-2 [56]	70 billion	12	7	4
GPT-3 [1]	175 billion	30	18	9
Grok-1 [59]	314 billion	53	32	16
LLaMA-3 [10]	405 billion	68	41	21

表 2. Google Compute Engine 上 6 个区域中不同 GPU 实例的可用性 [11]。

Region	GPU Type					
	H100	A100 80GB	A100 40GB	L4	T4	V100
us-central-1	✓	✓	✓	✓	✓	✓
us-east-4	✓	✓	×	✓	✓	×
us-east-1	×	×	✓	✓	✓	✓
eu-west-3	✓	×	×	✓	✓	×
asia-ne-1	✓	×	✓	✓	✓	×
asia-ne-3	×	×	✓	✓	✓	×

第 1 卷 (ASPLOS ’25)，2025 年 3 月 30 日至 4 月 3 日，荷兰鹿特丹。 ACM，美国纽约州纽约市，17 页。 https://doi.org/10.1145/3669940.3707215

1 简介

GPT-4 [1] 和 LLaMA-3 [28] 等生成式大语言模型 (LLM) 已经展示了在一系列应用领域创建自然语言文本的卓越能力，包括聊天机器人 [38]、编码助手 [26, 44] 和任务自动化 [15]。然而，现代大语言模型越大的模型尺寸和高计算要求使得在现代云平台上廉价而高效地为其提供服务变得具有挑战性。特别是，当今大多数 LLM 服务系统（例如 Orca [61] 和 vLLM [22]）都以同构 GPU 集群 [29] 为目标，其中所有 GPU 都是同一类型，并且具有相同的内存容量和计算资源。由于型号增加

Table 3. Properties of GPUs deployed in today's data centers. We report SXM version for H100 and A100. Data collected from NVIDIA data sheet [32–35].

GPU	FP16 (TFLOPs)	Memory (GB)	Bandwidth (GB/s)	Power (W)	Price (USD)
H100 [33]	1979	80	3350	700	25k ~40k
A100 [32]	312	40	1555	400	10k ~15k
L4 [34]	242	24	300	72	~3k
T4 [35]	65	16	300	70	~1k

Table 4. GPU quotas and deployment limits in us-east and asia-southeast during our evaluation of Helix. We measure deployment limits at two distinct time periods.

Region	Quota			Max Deployed		
	A100 40GB	L4	T4	A100 40GB	L4	T4
us-east	8	8	16	0	0	16
asia-southeast	8	24	32	4	12	>20

sizes, serving LLMs using homogeneous GPUs requires an increasing number of GPUs, as shown in Table 1. In addition, serving state-of-the-art LLMs used in industry requires even more resources. Recent works have identified that it is increasingly difficult to allocate GPUs of this magnitude within a single cloud region [50, 60].

Due to advances in GPU architectural designs and the incremental deployment of them over time, modern cloud platforms increasingly consist of a mix of GPU types. Table 2 illustrates the *heterogeneous* GPU deployment in Google Compute Engine [11], where datacenters are equipped with various NVIDIA GPUs including H100, A100, V100, L4, and T4. These heterogeneous GPU instances are spread across datacenters around the world and collectively offer significantly larger memory capacity and more compute resources than individual GPU types, enabling a more accessible and scalable approach to LLM serving. As Table 3 shows, eight NVIDIA L4 GPUs can offer comparable FP16 compute performance to a single NVIDIA H100 GPU, while providing greater memory capacity, lower power consumption, and a more cost-effective price point. Moreover, the availability of these GPUs vary significantly across regions. We empirically find that obtaining higher quotas and securing GPU instances is much easier in some regions than others on Google Cloud Engine, and the availability of different GPUs vary a lot (see Table 4).

Geo-distributed LLM serving with heterogeneous GPUs enables the aggregation of available GPUs from multiple regions. This approach not only enhances resource utilization but also minimizes LLM serving costs by strategically leveraging the most cost-effective GPU instances across various geographical locations. Similarly, there is also a trend of using volunteer consumer GPUs to address the GPU scarcity problem [9, 46, 62]. However, in contrast to homogeneous GPU instances, deploying LLMs on geo-distributed heterogeneous instances necessitates accommodating various GPU devices and network conditions.

Prior work has introduced several systems for running machine learning computation over heterogeneous devices or geo-distributed environments. However, prior attempts either focus on long-running training workloads [16, 27, 39, 45, 64], which cannot adapt to LLM serving scenarios with real-time inference requests, or focus on decentralized serving with volunteer computing [5, 6], which lack the global coordination necessary to efficiently use GPU and network resources in clusters.

To efficiently serve LLMs over heterogeneous GPUs and network, we propose Helix, a distributed system for high-throughput, low-latency LLM serving. Helix's key idea is to formulate the execution of LLM serving over heterogeneous GPUs and network as a *data flow* problem under the constraints of diverse GPU computing capabilities, memory capacities, as well as complex inter-GPU connections. Helix leverages mixed integer linear programming to determine optimal model placement under these constraints. To accommodate heterogeneity, Helix introduces *per-request pipelines*, where each request has its own independent pipeline for scheduling. This combination of flow-based formulation and per-request pipelines enables Helix to achieve high GPU utilization in heterogeneous and geo-distributed GPU clusters. We will discuss the challenges and Helix's solutions in Sec. 3.

We have implemented Helix on top of vLLM [22] and evaluated it on three heterogeneous clusters ranging from 24 to 42 nodes, with up to 7 different node types. The models we evaluated include LLaMA-1 30B and LLaMA-2 70B. Compared to heterogeneity-aware baselines, Helix improves serving throughput by up to 3.3× while reducing average prompting and decoding latency by up to 66% and 24%.

In summary, our contributions are:

- A system for LLM serving in heterogeneous and geo-distributed GPU clusters.
- A max-flow formulation for LLM serving and an MILP-based algorithm to optimize model placement.
- Flexible flow-based per-request pipelines to maximize GPU utilization.
- An implementation of our techniques and an evaluation on various LLM benchmarks.

2 Background

2.1 LLM Architecture and Serving

Most of today's LLMs adopt a decoder-only Transformer architecture [7, 42], which begins by converting a natural language query into a sequence of tokens. The model then converts each token into a hidden state vector, whose size is referred to as the model's hidden size. A Transformer model comprises of input and output embeddings and a series of identical Transformer layers, each consisting of a self-attention and a feed-forward block. A self-attention block

表 3. 当今数据中心中部署的 GPU 的属性。我们报告 H100 和 A100 的 SXM 版本。数据收集自 NVIDIA 数据表 [32-35]。

GPU	FP16 (TFLOPs)	Memory (GB)	Bandwidth (GB/s)	Power (W)	Price (USD)
H100 [33]	1979	80	3350	700	25k ~40k
A100 [32]	312	40	1555	400	10k ~15k
L4 [34]	242	24	300	72	~3k
T4 [35]	65	16	300	70	~1k

由于大小不同，使用同构 GPU 为 LLM 提供服务需要越来越多的 GPU，如表 1 所示。此外，为行业中使用的最先进的 LLM 提供服务需要更多的资源。最近的研究表明，在单个云区域内分配如此规模的 GPU 变得越来越困难 [50, 60]。

由于 GPU 架构设计的进步以及随着时间推移的增量部署，现代云平台越来越多地由多种 GPU 类型组成。表 2 说明了 Google 计算引擎 [11] 中的异构 GPU 部署，其中数据中心配备了各种 NVIDIA GPU，包括 H100、A100、V100、L4 和 T4。这些异构 GPU 实例分布在世界各地的数据中心，与单个 GPU 类型相比，它们共同提供更大的内存容量和更多的计算资源，从而为 LLM 服务提供更易于访问和扩展的方法。如表 3 所示，八个 NVIDIA L4 GPU 可以提供与单个 NVIDIA H100 GPU 相当的 FP16 计算性能，同时提供更大的内存容量、更低的功耗和更具成本效益的价格点。此外，这些 GPU 的可用性在不同地区存在很大差异。我们根据经验发现，在 Google Cloud Engine 上，某些区域比其他区域更容易获得更高的配额和保护 GPU 实例，并且不同 GPU 的可用性差异很大（参见表 4）。

使用异构 GPU 的地理分布式 LLM 可以聚合来自多个区域的可用 GPU。这种方法不仅提高了资源利用率，还通过战略性地利用不同地理位置的最具成本效益的 GPU 实例，最大限度地降低了 LLM 服务成本。同样，还有一种趋势是使用志愿消费 GPU 来解决 GPU 稀缺问题 [9, 46,62]。然而，与同构 GPU 实例相比，在地理分布式异构实例上部署 LLM 需要适应各种 GPU 设备和网络条件。

之前的工作已经介绍了几种用于在异构设备或地理分布式环境上运行机器学习计算的系统。然而，之前的尝试要么专注于长时间运行的训练工作负载[16,27,39,45,64]，这些工作负载无法适应具有实时推理请求的LLM服务场景，要么专注于去中心化

表 4. 我们评估 Helix 期间美国东部和亚洲东南部的 GPU 配额和部署限制。我们测量两个不同时间段的部署限制。

Region	Quota			Max Deployed		
	A100 40GB	L4	T4	A100 40GB	L4	T4
us-east	8	8	16	0	0	16
asia-southeast	8	24	32	4	12	>20

服务于志愿者计算 [5, 6]，缺乏有效使用集群中的 GPU 和网络资源所需的全局协调。

为了通过异构 GPU 和网络有效地为 LLM 提供服务，我们提出了 Helix，一种用于高吞吐量、低延迟 LLM 服务的分布式系统。 Helix 的关键思想是将在异构 GPU 和网络上服务的 LLM 的执行制定为在不同 GPU 计算能力、内存容量以及复杂的 GPU 间连接的约束下的数据流问题。 Helix 利用混合整数线性规划来确定这些约束下的最佳模型放置。为了适应异构性，Helix 引入了每个请求管道，其中每个请求都有自己独立的管道进行调度。基于流的制定和按请求管道的结合使 Helix 能够在异构和地理分布式 GPU 集群中实现高 GPU 利用率。我们将在第二部分讨论挑战和 Helix 的解决方案。 3.

我们在 vLLM [22] 之上实现了 Helix，并在 24 到 42 个节点、最多 7 种不同节点类型的三个异构集群上对其进行了评估。我们评估的模型包括 LLaMA-1 30B 和 LLaMA-2 70B。与异构感知基线相比，Helix 将服务吞吐量提高了高达 3.3×，同时将平均提示和解码延迟降低了高达 66% 和 24%。

总之，我们的贡献是：

- 一种服务于异构和地理分布式 GPU 集群的 LLM 系统。
- 用于 LLM 服务的最大流量公式和用于优化模型放置的基于 MILP 的算法。
- 灵活的基于流的每个请求管道可最大限度地提高 GPU 利用率。
- 我们的技术的实施以及对各种 LLM 基准的评估。

2 背景

2.1 LLM架构和服务

当今大多数大语言模型用仅解码器的 Transformer 架构 [7, 42]，该架构首先将自然语言查询转换为标记序列。然后，模型将每个标记转换为隐藏状态向量，其大小称为模型的隐藏大小。 Transformer 模型由输入和输出嵌入以及一系列相同的 Transformer 层组成，每个层都包含一个自注意力和一个前馈块。自注意力块

calculates the 'affinity' between every pair of tokens and updates each token's hidden states based on this contextual relevance score. Feed-forward blocks independently modify each token's hidden state through a non-linear function.

Given an input sequence, a Transformer model computes the probability distribution for the next token, and samples from this probability distribution. Thus, the model applies an auto-regressive paradigm to generate the whole output sequence: given an *input prompt*, a model runs multiple iterations. At the first iteration, known as the *prompt phase*, the model processes all prompt tokens and generates the first output token. In subsequent iterations, known as the *decode phase*, the model incorporates both prompt and previously generated tokens to predict the next output token. This iterative process stops when model produces a special end-of-sentence signal (⟨eos⟩). Since the generation output is unpredictable, the exact number of iterations remains uncertain until the sequence is fully generated.

In addition to the unpredictable execution iterations, another feature of LLM serving is the high memory demand. The self-attention block requires all previous tokens' hidden states as inputs. To store the hidden states (known as the KV-cache) for newly generated tokens, the memory requirements keep increasing along the generation process.

To address these challenges, Orca [61] presented iteration-level scheduling, which updates a batch at every iteration to avoid resource retention when a request is completed but others in the same batch need more iterations; vLLM [22] introduces PagedAttention, managing memory for KV-cache with identical pages and allocating a new page only when a request has used up all its pages; multi-query [47] and group-query attention [3] modifies the self-attention mechanism to reduce the size of KV-cache stored for each token.

2.2 Distributed Model Serving

Open source LLMs now feature up to hundreds of billions of parameters, far exceeding the memory capacity of a single GPU. Consequently, serving an LLM requires multiple GPUs operating in parallel. Tensor Parallelism (TP) [49] partitions the weight of each operator among GPUs, gathering the partial results on each device via an AllReduce/AllGather operation. However, TP is highly sensitive to network conditions. For every Transformer layer, it needs two communications. As a result, TP has a significant overhead in high-latency networks, and is only used among GPUs within a node.

Conversely, Pipeline Parallelism (PP) [17] assigns different operators (typically multiple layers) across GPUs to create multiple pipeline stages. It then splits inputs into micro-batches, running them through the pipeline. PP only transmits the activation tensor at the boundary of pipeline stages. Hence, PP is much less network-sensitive. However, it is challenging to perfectly partition both the model and input batch, which results in pipeline bubbles. As a result, PP suffers from the device idle at pipeline bubbles, and necessitating careful schedule to be performant [2].

Traditional data center setups typically assume *homogeneous* clusters: uniform nodes with a uniform bandwidth. As a result, models are *evenly* partitioned into pipeline stages and assigned to each devices. As LLMs grow in size and the latest generation GPUs remain scarce, deploying these models across heterogeneous computing devices has become a critical necessity, a challenge that previous research has not adequately addressed.

Deploying LLMs across geo-distributed, heterogeneous GPU clusters requires careful consideration of both hardware capabilities and network characteristics. Simple equal distribution of model layers across devices fails to maximize the potential of more powerful hardware. Moreover, the significant bandwidth differences between intra- and inter-regional network connections must inform both model placement and request scheduling decisions, particularly when infrastructure spans multiple geographical regions.

No prior work has focused on LLM serving on heterogeneous GPU clusters. The most related work is Petals [5], which performs decentralized LLM serving with volunteer computing. Users contribute GPUs to form a decentralized swarm and newly joined machines greedily serve the pipeline stages with least compute capacity. For request scheduling, users greedily choose the server with lowest latency to them. Petals is effective for volunteer computing, but the lack of global coordination makes it unable to fully utilize the GPUs and network when the cluster is a known priori. Another relevant work is SWARM [45], which performs DNN training in heterogeneous clusters. It evenly partitions the model into pipeline stages. When routing a request to the next pipeline stage, it selects the replica based on real-time throughput of candidates. Our evaluation shows that such simple heuristics are not enough to achieve good performance in geo-distributed clusters with heterogeneous GPUs and network.

3 Opportunities and Challenges

Using heterogeneous and geo-distributed GPUs presents new opportunities for LLM serving. As shown in Table 3, multiple commodity GPUs (L4, T4) can match the compute capacity of high-end GPUs (H100, A100) while offering advantages in memory capacity, energy efficiency, and cost. Furthermore, Tables 4 and 7 demonstrate that GPU availability varies significantly across regions, yet inter-region network conditions remain suitable for LLM serving, motivating a geo-distributed approach. However, leveraging heterogeneous and geo-distributed GPUs poses several key challenges.

3.1 Challenge 1: Model Placement

Due to the increasing size of LLMs, serving them on modern GPUs requires employing *tensor* [49] and *pipeline* [17, 39]

计算每对标记之间的“亲和力”，并根据上下文相关性得分更新每个标记的隐藏状态。前馈块通过非线性函数独立修改每个令牌的隐藏状态。

给定输入序列，Transformer 模型计算下一个标记的概率分布，并从该概率分布中采样。因此，该模型应用自回归范例来生成整个输出序列：给定输入提示，模型运行多次迭代。在第一次迭代（称为提示阶段）时，模型处理所有提示标记并生成第一个输出标记。在随后的迭代（称为解码阶段）中，模型结合提示和之前生成的标记来预测下一个输出标记。当模型产生特殊的句尾信号 (⟨eos⟩) 时，此迭代过程停止。由于生成输出是不可预测的，因此在序列完全生成之前，确切的迭代次数仍然不确定。

除了不可预测的执行迭代之外，LLM 服务的另一个特点是高内存需求。自注意力块需要所有先前令牌的隐藏状态作为输入。为了存储新生成的令牌的隐藏状态（称为 KV 缓存），内存需求随着生成过程不断增加。

为了应对这些挑战，Orca [61]提出了迭代级调度，它在每次迭代时更新一个批次，以避免当请求完成但同一批次中的其他人需要更多迭代时资源保留； vLLM [22]引入了 PagedAttention，为具有相同页面的 KV 缓存管理内存，并仅在请求用完其所有页面时才分配新页面；多查询[47]和组查询注意[3]修改了自注意机制以减少为每个令牌存储的KV缓存的大小。

2.2 分布式模型服务

开源 LLM 现在具有多达数千亿个参数，远远超过单个 GPU 的内存容量。因此，为大语言模型服务需要多个 GPU 并行运行。张量并行性（TP）[49]在 GPU 之间划分每个算子的权重，通过 AllReduce/AllGather 操作收集每个设备上的部分结果。然而，TP对网络状况非常敏感。对于每个 Transformer 层，它需要两次通信。因此，TP 在高延迟网络中具有显着的开销，并且仅在节点内的 GPU 之间使用。

相反，管道并行性 (PP) [17] 在 GPU 上分配不同的运算符（通常是多个层）以创建多个管道阶段。然后它将输入分成微批次，并通过管道运行它们。 PP 仅在管道阶段的边界传输激活张量。因此，PP 对网络的敏感度要低得多。然而，完美划分模型和输入批次具有挑战性，这会导致管道气泡。结果，PP 遭受了设备在管道出现气泡时闲置，需要仔细安排才能提高性能 [2]。

传统的数据中心设置通常假设同质集群：具有统一带宽的统一节点。因此，模型被均匀地划分为管道阶段并分配给每个设备。随着大语言模型的不断扩大，而最新一代 GPU 仍然稀缺，跨异构计算设备部署这些模型已成为一项至关重要的任务，而之前的研究尚未充分解决这一挑战。

跨地理分布式、异构 GPU 集群部署 LLM 需要仔细考虑硬件功能和网络特性。跨设备简单地平等分配模型层无法最大限度地发挥更强大硬件的潜力。此外，区域内和区域间网络连接之间的显着带宽差异必须为模型放置和请求调度决策提供信息，特别是当基础设施跨越多个地理区域时。

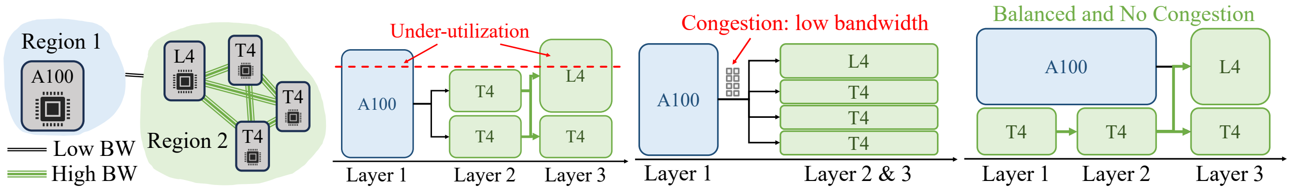
之前没有任何工作关注在异构 GPU 集群上服务的 LLM。最相关的工作是 Petals [5]，它通过志愿者计算执行去中心化的 LLM 服务。用户贡献 GPU 来形成分散的群，新加入的机器贪婪地以最少的计算能力服务于管道阶段。对于请求调度，用户贪婪地选择对他们来说延迟最低的服务器。 Petals对于志愿计算是有效的，但是缺乏全局协调使得在集群先验已知的情况下无法充分利用GPU和网络。另一个相关工作是 SWARM [45]，它在异构集群中进行 DNN 训练。它将模型均匀地划分为管道阶段。当将请求路由到下一个管道阶段时，它会根据候选者的实时吞吐量选择副本。我们的评估表明，这种简单的启发式方法不足以在具有异构 GPU 和网络的地理分布式集群中实现良好的性能。

3 机遇与挑战

使用异构和地理分布式 GPU 为 LLM 服务提供了新的机会。如表 3 所示，多种商用 GPU（L4、T4）可以与高端 GPU（H100、A100）的计算能力相匹配，同时在内存容量、能源效率和成本方面具有优势。此外，表 4 和表 7 表明，GPU 可用性在不同区域之间存在显着差异，但区域间网络条件仍然适合 LLM 服务，从而激发了地理分布式方法。然而，利用异构和地理分布式 GPU 带来了几个关键挑战。

3.1 挑战一：模型放置

由于 LLM 的规模不断增加，在现代 GPU 上为它们提供服务需要使用张量 [49] 和管道 [17, 39]



(a) Five nodes in two clusters. **(b)** Uniform model partition. **(c)** Balanced FLOPs. **(d)** Network aware placement.

Figure 1. Examples of sub-optimal model placement and request schedule. 1a) all GPUs and network condition in this example. The order of compute capacity is: A100 > L4 > T4; 1b) Model placement by uniformly partition the model, then allocate devices by a balanced compute capacity; 1c) Co-optimizing model partition and device placement to make the compute capacity more balanced; 1d) Co-optimizing model partition, device placement, and request scheduling in a network-aware way.

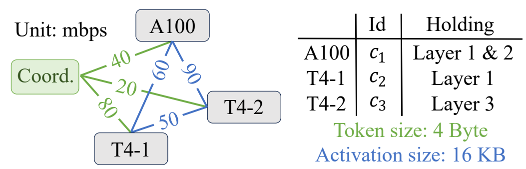
model parallelism to partition an LLM into stages and place those stages on different GPUs, a task we term *model placement*. Homogeneous serving systems (e.g., Orca [61]) partition an LLM into equal-sized stages and assign them to GPUs. This approach results in sub-optimal utilization of high-performance GPUs as it accommodates the memory and computational limitations of less powerful GPUs. Existing heterogeneity-aware serving systems (e.g., Petals [5]) rely on different heuristics to partition a model into stages and assign them to GPUs. Existing heuristics do not simultaneously consider both GPU and network heterogeneity.

Helix's solution: Helix exploits the flexibility of token-level scheduling in LLM serving and formulates model placement as a *max-flow* problem of a directed, weighted graph, whose nodes represent GPU instances and edges capture both GPU and network heterogeneity through their capacities in the max-flow problem. Helix then uses a mixed integer linear programming (MILP) algorithm to discover highly optimized model placement strategies, which largely outperform the heuristic methods used in prior work [5, 45]. Leveraging the data dependencies and homogeneity of LLM layers, Helix expresses the MILP problem with linear number of variables and constraints relative to the number of compute nodes and network connections, resulting in a tractable problem size.

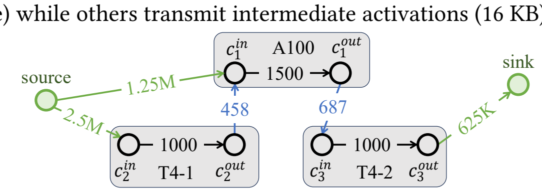
3.2 Challenge 2: Request Scheduling

A second challenge Helix must address is *request scheduling*. To serve an LLM request, Helix needs to select a *pipeline* of GPU instances to compute all layers of the LLM. Existing systems generally employ a group of fixed pipelines and assign requests to these pipelines in a round-robin fashion. Using fixed pipelines is not flexible enough to accommodate the heterogeneous compute and network conditions and often causes under-utilization.

Helix's solution: Helix introduces *per-request pipelines*, where each request is assigned its own pipeline. As a result, the total number of potential pipelines is equal to the number of paths from source to sink in the graph representation of the cluster, which offers sufficient flexibility for Helix to maximally utilize the full capacity of GPU instances and network connections between them.



(a) A 3-node cluster with model placement. Network connections between the coordinator and compute nodes transmit tokens (4 Byte) while others transmit intermediate activations (16 KB)



(b) Graph abstraction of the cluster.

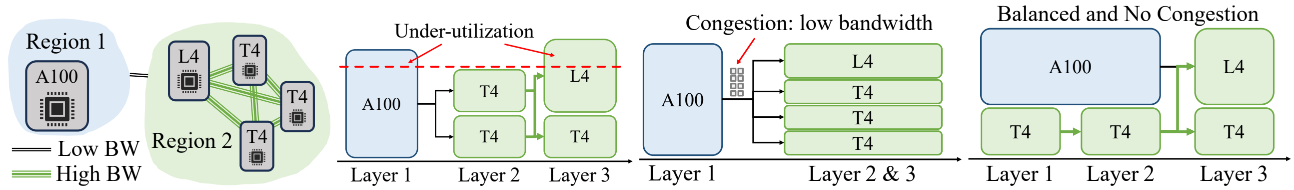
Figure 2. Graph abstraction of a 3-node cluster with given model placement. Numbers on the edges in Fig. 2b represent their capacity, which is the number of tokens that can pass through the edges per second. Max flow between source and sink equals the max serving throughput of the cluster.

4 Optimization Formulation in Helix

This section first provides a mathematical abstraction for LLM serving systems. Based on this formulation, we model heterogeneous LLM serving as a Max-Flow problem. Finally, we apply mixed-integer linear programming (MILP) to search for a model placement strategy with the highest max flow.

4.1 Formulation of LLM Serving

A cluster to serve LLMs generally contains one coordinator node h and a group of compute nodes C. Each compute node $c_i \in C$ has a compute capacity and GPU VRAM size. Compute nodes with multiple GPUs can be abstracted as a single logical node, aggregating GPUs' combined computational capacity and GPU VRAM resources. Throughput and latency of network connections between nodes in the cluster are also given. Based on the cluster information, LLM serving requires finding a placement of model layers to compute nodes to maximize a *serving performance metric* that will be defined below. The model placement function $\Psi : C \mapsto \mathcal{P}(\mathcal{M})$ takes



(a) Five nodes in two clusters. **(b)** Uniform model partition. **(c)** Balanced FLOPs. **(d)** Network aware placement.

图 1.次优模型放置和请求计划的示例。 1a) 本例中的所有 GPU 和网络状况。计算能力顺序为：A100 > L4 > T4； 1b ）通过对模型进行统一分区来进行模型放置，然后通过平衡计算能力来分配设备； 1c) 协同优化模型划分和设备布局，使计算能力更加均衡； 1d) 以网络感知的方式共同优化模型分区、设备布局和请求调度。

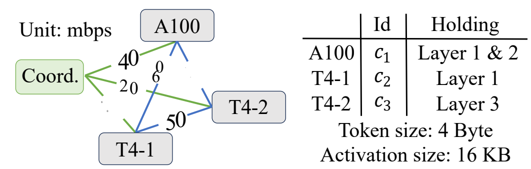
模型并行性将 LLM 划分为多个阶段并将这些阶段放置在不同的 GPU 上，我们将这一任务称为模型放置。同质服务系统（例如 Orca [61]）将 LLM 划分为大小相等的阶段并将它们分配给 GPU。这种方法会导致高性能 GPU 的利用率不佳，因为它适应了功能较弱的 GPU 的内存和计算限制。现有的异构感知服务系统（例如 Petals [5]）依靠不同的启发式方法将模型划分为多个阶段并将其分配给 GPU。现有的启发式方法并未同时考虑 GPU 和网络异构性。

Helix 的解决方案：Helix 利用 LLM 服务中令牌级调度的灵活性，并将模型放置制定为有向加权图的最大流问题，其节点代表 GPU 实例，边通过其在最大流问题中的能力捕获 GPU 和网络异构性。然后，Helix 使用混合整数线性规划 (MILP) 算法来发现高度优化的模型放置策略，该策略在很大程度上优于先前工作中使用的启发式方法 [5, 45]。利用LLM层的数据依赖性和同质性，Helix用相对于计算节点和网络连接数量的线性变量和约束来表达MILP问题，从而产生易于处理的问题大小。

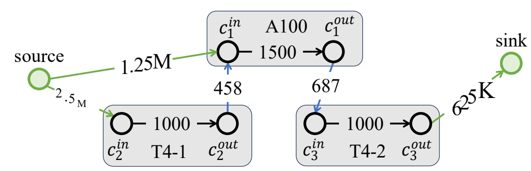
3.2 挑战2：请求调度

Helix 必须解决的第二个挑战是请求调度。为了满足 LLM 请求，Helix 需要选择 GPU 实例管道来计算 LLM 的所有层。现有系统通常采用一组固定管道，并以循环方式将请求分配给这些管道。使用固定管道不够灵活，无法适应异构计算和网络条件，并且经常导致利用率不足。

Helix 的解决方案：Helix 引入了按请求管道，其中每个请求都分配有自己的管道。因此，潜在管道的总数等于集群图形表示中从源到接收器的路径数量，这为Helix提供了足够的灵活性，可以最大限度地利用GPU实例的全部容量以及它们之间的网络连接。



(a) 具有模型放置的 3 节点集群。协调器和计算节点之间的网络连接传输令牌（4 字节），而其他节点则传输中间激活（16 KB）



(b) Graph abstraction of the cluster.

图 2. 具有给定模型放置的 3 节点集群的图形抽象。图2b中边缘上的数字代表其容量，即每秒可以通过边缘的令牌数量。源和宿之间的最大流量等于集群的最大服务吞吐量。

4 Helix 中的优化公式

本节首先提供 LLM 服务系统的数学抽象。基于这个公式，我们将异构 LLM 建模为最大流问题。最后，我们应用混合整数线性规划（MILP）来搜索具有最高最大流量的模型放置策略。

4.1 LLM服务的制定

服务LLM的集群通常包含一个协调器节点h和一组计算节点C。每个计算节点$c_i \in C$都有计算能力和GPU VRAM大小。具有多个GPU的计算节点可以抽象为单个逻辑节点，聚合GPU的组合计算能力和GPU VRAM资源。还给出了集群中节点之间网络连接的吞吐量和延迟。基于集群信息，LLM 服务需要找到模型层到计算节点的位置，以最大化下面将定义的服务性能指标。模型放置函数 $\Psi : C \mapsto \mathcal{P}(M)$ 采用

as input a compute node and returns a (usually continuous) subset of the model $\mathcal{M}$. Here, we assume pipeline parallelism for inter-node parallelization and tensor parallelism for intra-node parallelization across GPUs, as tensor parallelism demands extensive communication and may be constrained by network. One widely used metric [5, 45] to assess a model placement is the performance of the pipeline stage with the lowest compute capacity $\min_i \sum_j \text{capacity}(j) \cdot 1_{i \in \Psi(j)}$, where $\text{capacity}(j)$ is the compute capacity of the j^{th} node. As we will show below, only considering compute capacity yields sub-optimal model placements for heterogeneous clusters.

Based on model placement, LLM serving requires a request scheduling strategy that can efficiently serve requests in the cluster. The request scheduling strategy $\phi : \mathcal{R} \mapsto C^k$ inputs a request and outputs a sequence of compute nodes that form a complete pipeline for executing all layers of the LLM.

4.2 Necessity of Joint Optimization

Before diving into our Max-Flow formulation of heterogeneous LLM serving, we first use an example to show why we need to co-optimize model partition, device placement, and request scheduling as a Max-Flow problem. In this example, the clusters are shown in Fig. 1a. There are two regions with a low bandwidth between them. Region 1 has a powerful A100 GPU, while Region 2 has a less powerful L4 GPU and three T4 GPUs, but has a high bandwidth within the region. The pairwise bandwidths are independent. If we follow the common approach, which statically partitions the model and then assigns devices to each partition, the placement plan will be as Fig. 1b. In this plan, although the last pipeline stage has a T4 and an L4 GPU, its throughput is bound by the previous stage's output throughput, which only has 2 T4 GPUs. This indicates a necessity to co-optimize the pipeline partition plan and placement of pipeline stages.

However, even with a perfectly balanced compute capacity at each pipeline stage, as shown in Fig. 1c, the solution can still be sub-optimal. In this solution, it assigns the powerful A100 to individually serve some layers, while other GPUs run in a data parallel manner for the rest of the layers. However, communications from one pipeline stage to another become a bottleneck. For every request, its intermediate state is sent from Region 1 to Region 2 via low bandwidth. This eventually creates congestion on the A100's send side. Instead, Fig. 1d assigns two T4 GPUs running in parallel with the A100. This divides the workload between the A100 GPU and the two T4 GPUs, reducing communication on the slow link.

4.3 Heterogeneous LLM Serving as Max-Flow

To optimize model placement, Helix needs a way to determine the max serving throughput of different model placements. To achieve this, we transform a cluster of compute nodes with assigned model layers into a directed graph with edge capacity. The edge capacity denotes the number of tokens compute nodes and network connections can process/transmit per second. Max flow between source and sink vertices in the graph, which represent the coordinator node, gives us the max serving throughput of the cluster with the current model placement. The following shows the formal construction.

Table 5. Variables used in MILP.

Symbol	Type	Num.	Description
s_i	int	$O(\|C\|)$	index of c_i's first layer
b_i^j	binary	$O(\|C\|)$	whether c_i holds j layers
$f_{i,j}$	real	$O(\|\mathcal{E}\|)$	flow from c_i to c_j
$d_{i,j}$	binary	$O(\|\mathcal{E}\|)$	whether (c_i, c_j) is valid
$cond_{i,j}^1$	binary	$O(\|\mathcal{E}\|)$	aux. variable in constraint-4
$cond_{i,j}^2$	binary	$O(\|\mathcal{E}\|)$	aux. variable in constraint-4

For a given cluster with coordinator node h, a set of compute nodes C, and a model placement Ψ, we can transform entities in the cluster into elements of its graph abstraction as follows. An example of such a graph abstraction of a cluster with *given model placement* is shown in Fig. 2

Compute and coordinator nodes. For each compute node $c_i \in C$, we represent it with two connected vertices in the graph. We name the two vertices c_i^{in} and c_i^{out}. The capacity of the directed edge (c_i^{in}, c_i^{out}) represents the max number of tokens this node can process in one second. It is the minimum of the node's compute and network throughput. Helix performs a one-time profiling to measure the throughput of all compute nodes. For the coordinator node, we represent it as source and sink vertices in the graph.

Network connections. In a given cluster, a node may communicate with any other nodes, creating $O(|C|^2)$ possible directed network connections between different nodes. However, only a subset of those connections are valid based on the model placement as described below. A *valid connection* should satisfy one of the following three criteria: (1) the connection is from coordinator node h to compute node c_i and c_i holds the first layer of the model; (2) the connection is from a compute node c_j to coordinator node h and c_j holds the last layer of the model; (3) the connection is from one compute node c_i to another compute node c_j and c_j holds model layers immediately needed after inference on c_i. For the first and second case, we represent the connection with directed edge $(source, c_i^{in})$ and $(c_j^{out}, sink)$ respectively, with capacity equal to the connection bandwidth divided by the transmission size of a token (a few bytes). For the third case, we represent the connection with a directed edge (c_i^{out}, c_j^{in}), and the capacity equals the connection bandwidth divided by the transmission size of an activation (tens of kilobytes). Helix performs a one-time profiling and uses the average bandwidth as the connection bandwidth. The capacity of the edges models the throughput constraint imposed by the speed of network connection between different nodes. We denote the full set of possible network connections by $\mathcal{E}$.

作为输入计算节点并返回模型 $\mathcal{M}$ 的（通常是连续的）子集。在这里，我们假设跨 GPU 的节点间并行化采用管道并行化，而跨 GPU 的节点内并行化采用张量并行化，因为张量并行化需要广泛的通信，并且可能受到网络的限制。评估模型放置的一种广泛使用的指标 [5, 45] 是具有最低计算能力 $\min_i \sum_j$ 容量$(j) \cdot 1_{i \in \Psi(j)}$ 的管道阶段的性能，其中容量(j) 是 j^{th} 节点的计算能力。正如我们将在下面展示的，仅考虑计算容量会导致异构集群的次优模型放置。

基于模型放置，LLM 服务需要一个能够高效服务集群中请求的请求调度策略。请求调度策略ϕ: $\mathcal{R} \mapsto \mathcal{C}^k$输入请求并输出一系列计算节点，这些计算节点形成执行LLM所有层的完整管道。

4.2 联合优化的必要性

在深入研究异构 LLM 服务的 Max-Flow 公式之前，我们首先使用一个示例来说明为什么我们需要将模型分区、设备放置和请求调度作为 Max-Flow 问题进行协同优化。在此示例中，簇如图 1a 所示。有两个区域之间的带宽较低。区域 1 拥有强大的 A100 GPU，而区域 2 拥有性能稍弱的 L4 GPU 和 3 个 T4 GPU，但该区域内的带宽较高。成对带宽是独立的。如果我们遵循常见的方法，即对模型进行静态分区，然后将设备分配给每个分区，则布局计划将如图 1b 所示。在这个计划中，虽然最后一个管道阶段有一个T4和一个L4 GPU，但它的吞吐量受到前一个阶段的输出吞吐量的限制，而前一个阶段只有2个T4 GPU。这表明有必要共同优化管道分区计划和管道阶段的放置。

然而，即使每个管道阶段的计算能力完全平衡，如图 1c 所示，解决方案仍然可能不是最优的。在此解决方案中，它分配强大的 A100 单独服务某些层，而其他 GPU 以数据并行方式运行其余层。然而，从一个管道阶段到另一个管道阶段的通信成为瓶颈。对于每个请求，其中间状态通过低带宽从区域 1 发送到区域 2。这最终会在 A100 的发送端造成拥塞。相反，图 1d 分配了两个与 A 100 并行运行的 T4 GPU。这会在 A100 GPU 和两个 T4 GPU 之间分配工作负载，从而减少慢速链路上的通信。

4.3 异构LLM作为最大流

为了优化模型放置，Helix 需要一种方法来确定不同模型放置的最大服务吞吐量。为了实现这一目标，我们将具有指定模型层的计算节点集群转换为具有边缘容量的有向图。边容量表示边的数量

表 5. MILP 中使用的变量。

Symbol	Type	Num.	Description
s_i	int	$O(\|\mathcal{C}\|)$	index of c_i's first layer
b_i^j	binary	$O(\|\mathcal{C}\|)$	whether c_i holds j layers
$f_{i,j}$	real	$O(\|\mathcal{E}\|)$	flow from c_i to c_j
$d_{i,j}$	binary	$O(\|\mathcal{E}\|)$	whether (c_i, c_j) is valid
$cond_{i,j}^1$	binary	$O(\|\mathcal{E}\|)$	aux. variable in constraint-4
$cond_{i,j}^2$	binary	$O(\|\mathcal{E}\|)$	aux. variable in constraint-4

令牌计算节点和网络连接每秒可以处理/传输。图中源顶点和汇顶点之间的最大流量（代表协调器节点）为我们提供了当前模型放置的集群的最大服务吞吐量。下图展示了正式的构建。

对于具有协调器节点 h、一组计算节点 $\mathcal{C}$ 和模型放置 Ψ 的给定集群，我们可以将集群中的实体转换为其图形抽象的元素，如下所示。图 2 显示了具有给定模型放置的集群的图形抽象示例。

计算节点和协调节点。对于每个计算节点 $c_i \in \mathcal{C}$，我们用图中的两个连接的顶点来表示它。我们将两个顶点命名为 c_i^{in} 和 c_i^{out}。有向边(c_i^{in}, c_i^{out})的容量表示该节点一秒内可以处理的最大令牌数。它是节点计算和网络吞吐量的最小值。 Helix 执行一次性分析来测量所有计算节点的吞吐量。对于协调器节点，我们将其表示为图中的源顶点和汇顶点。

网络连接。在给定的集群中，节点可以与任何其他节点进行通信，从而在不同节点之间创建$O(|\mathcal{C}|^2)$可能的定向网络连接。然而，根据如下所述的模型放置，这些连接中只有一部分是有效的。有效的连接应满足以下三个标准之一：（1）连接是从协调器节点h到计算节点c_i，并且c_i保存模型的第一层； (2) 连接是从计算节点 c_j 到协调器节点 h，并且 c_j 保存模型的最后一层； (3) 连接是从一个计算节点 c_i 到另一计算节点 c_j 的连接，并且 c_j 保存在推理 c_i 后立即需要的模型层。对于第一种和第二种情况，我们分别表示与有向边$(source, c_i^{in})$和$(c_j^{out}, sink)$的连接，其容量等于连接带宽除以令牌的传输大小（几个字节）。对于第三种情况，我们用有向边 (c_i^{out}, c_j^{in}) 表示连接，容量等于连接带宽除以激活的传输大小（数十千字节）。 Helix 执行一次性分析并使用平均带宽作为连接带宽。边缘的容量对不同节点之间的网络连接速度所施加的吞吐量约束进行建模。我们用 $\mathcal{E}$ 表示完整的可能网络连接集。

Table 6. Constraints used in MILP.

Group	Num.	Constraint
Model placement	$O(\|C\|)$	$\sum_{j=1}^{k} b_i^j = 1$
	$O(\|C\|)$	$0 \le s_i < L$ and $e_i \le L$
Flow conservation	$O(\|C\|)$	$\sum_u f_{u,i} = \sum_v f_{i,v}$
Infer. throughput	$O(\|C\|)$	$\sum_u f_{ui} \le \sum_{j=1}^{k} b_i^j \cdot T_j$
Connection validity	$O(\|C\|)$	$s_i \le L \cdot (1 - d_{source,i})$
	$O(\|C\|)$	$L \cdot d_{i,sink} \le e_i$
	$O(\|\mathcal{E}\|)$	$(L+1)(1 - cond_{i,j}^1) \ge s_j - e_i$
	$O(\|\mathcal{E}\|)$	$e_j - e_i \ge 1 - (L+1)(1 - cond_{i,j}^2)$
	$O(\|\mathcal{E}\|)$	$d_{i,j} \le 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$
Trans. throughput	$O(\|\mathcal{E}\|)$	$f_{i,j} \le d_{i,j} \cdot S_{i,j}$

After constructing the equivalent graph abstraction of a cluster, we run the preflow-push algorithm [8] to get the max flow between source and sink node. One unit of flow here represents one token that can pass through a compute node or network connection in one second. Therefore, the max flow gives us the max possible serving throughput of the cluster with current model placement.

4.4 Optimal Model Placement with MILP

The previous section presented an approach for obtaining the max serving throughput of a cluster *for a given model placement*. In this section, we introduce a mixed-integer linear programming (MILP)-based method to find a model placement that maximizes the max flow, thus maximizing serving throughput. The MILP formulation has a linear number of variables and constraints with respect to the number of compute nodes and network connections. The key challenges addressed include (1) formulation of system-level constraints as linear number of conditions to satisfy, (2) expression of these conditions with linear number of variables, and (3) linearization of each condition using auxiliary variables, specifically, each constraint is expressed as at most three linear constraints with the help of at most two auxiliary variables. An overview of the variables and constraints is shown in Table 5 and 6.

Node variables. To represent the model placement on each compute node, we introduce two groups of variables in our MILP formulation. Suppose that the model has a total of L layers and each compute node holds a continuous subset of the model. For each compute node c_i, we introduce an integer variable s_i to represent the first layer c_i holds. Suppose compute node c_i can hold at most k layers on its GPU, we further introduce k binary variables $b_i^1, b_i^2, ..., b_i^k$ to indicate the number of layers node c_i holds ($b_i^j = 1$ if c_i holds j layers). We choose to express model placement with k binary variables (instead of one integer for the number of layers) because this formulation facilitates the expression of inference throughput constraints as discussed below. The *end layer index* of c_i can be expressed as $e_i = s_i + \sum_{j=1}^{k} j \cdot b_i^j$. Therefore, c_i holds layers in range $[s_i, e_i)$.

Connection variables. We introduce two groups of variables to constrain the number of inference requests that can go through each network connection. For network connection between compute node c_i and c_j, we introduce a real variable $f_{i,j}$ to denote the amount of flow from c_i^{out} to c_j^{in} in the graph abstraction. We further introduce a binary variable $d_{i,j}$ to denote whether the network connection is valid (as defined in Sec. 4.3). The constraints we introduce below will use $d_{i,j}$ to ensure that requests can only be transmitted through valid connections. For network connections between coordinator node and compute nodes, we similarly introduce two variables similar to above, but replace i/j with source/sink.

Constraint-1: model placement. To ensure that the model placement found by the MILP solver is valid, we need the following two constraints for each compute node c_i. First, c_i should have only one valid model placement, meaning that $\sum_{j=1}^{k} b_i^j = 1$. Moreover, the first and last layer c_i holds must be within the range of L layers, meaning that $0 \le s_i < L$ and $e_i \le L$. (c_i holds layers in range $[s_i, e_i)$)

Constraint-2: flow conservation. For each compute node c_i, the sum of flow that goes in to c_i^{in} must be equal to that goes out of c_i^{out} because of flow conservation. This constraint can be expressed as $\sum_u f_{u,i} = \sum_v f_{i,v}$, where u and v enumerate through all nodes except i.

Constraint-3: inference throughput. For compute node c_i, the amount of flow that passes through (c_i^{in}, c_i^{out}) should not exceed its maximum inference throughput. We can impose this constraint with $\sum_u f_{ui} \le \sum_{j=1}^{k} b_i^j \cdot T_j$. Here, T_j is a constant that represents the maximum number of tokens node c_i can process in one second when holding j layers, which is obtained through a one-time profiling process.

Constraint-4: connection validity. We need to determine the validity of network connections to know if requests can be transmitted through them. For a network connection from the coordinator node to the compute node c_i, it is valid only if c_i holds the first layer of the model. To express this constraint with MILP, we need to linearize it into the following form: $s_i \le L \cdot (1 - d_{source,i})$. Similarly, for network connection from compute node c_i to coordinator, we constrain its validity with $L \cdot d_{i,sink} \le e_i$. For network connection from compute node c_i to c_j, its validity $d_{i,j}$ is determined by whether $s_j \le e_i < e_j$ holds. To linearize this condition, we need to introduce two binary auxiliary variables $cond_{i,j}^1$ and $cond_{i,j}^2$. $cond_{i,j}^1$ takes value 1 only if $s_j \le e_i$, which can be linearized as $(L+1)(1 - cond_{i,j}^1) \ge s_j - e_i$. $cond_{i,j}^2$ takes value 1 only if $e_i < e_j$, which can be linearized as $e_j - e_i \ge 1 - (L+1)(1 - cond_{i,j}^2)$. The network connection is valid only if both binary auxiliary variables are true, which can be expressed as $d_{i,j} \le 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$.

表 6. MILP 中使用的约束。

Group	Num.	Constraint
Model placement	$O(\|C\|)$	$\sum_{j=1}^{k} b_i^j = 1$
	$O(\|C\|)$	$0 \le s_i < L$ and $e_i \le L$
Flow conservation	$O(\|C\|)$	$\sum_u f_{u,i} = \sum_v f_{i,v}$
Infer. throughput	$O(\|C\|)$	$\sum_u f_{ui} \le \sum_{j=1}^{k} b_i^j \cdot T_j$
Connection validity	$O(\|C\|)$	$s_i \le L \cdot (1 - d_{source,i})$
	$O(\|C\|)$	$L \cdot d_{i,sink} \le e_i$
	$O(\|\mathcal{E}\|)$	$(L+1)(1 - cond_{i,j}^1) \ge s_j - e_i$
	$O(\|\mathcal{E}\|)$	$e_j - e_i \ge 1 - (L+1)(1 - cond_{i,j}^2)$
	$O(\|\mathcal{E}\|)$	$d_{i,j} \le 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$
Trans. throughput	$O(\|\mathcal{E}\|)$	$f_{i,j} \le d_{i,j} \cdot S_{i,j}$

在构建了集群的等效图抽象之后，我们运行预流推送算法[8]来获得源节点和汇节点之间的最大流量。这里的一个流量单位代表一秒内可以通过计算节点或网络连接的一个令牌。因此，最大流量为我们提供了当前模型放置的集群的最大可能服务吞吐量。

4.4 MILP 的最佳模型放置

上一节介绍了一种获取给定模型放置的集群最大服务吞吐量的方法。在本节中，我们介绍一种基于混合整数线性规划（MILP）的方法来找到最大化最大流量的模型放置，从而最大化服务吞吐量。 MILP 公式具有线性数量的变量和关于计算节点和网络连接数量的约束。解决的关键挑战包括（1）将系统级约束表述为要满足的线性数量的条件，（2）用线性数量的变量表达这些条件，以及（3）使用辅助变量对每个条件进行线性化，具体来说，每个约束在最多两个辅助变量的帮助下表示为最多三个线性约束。表 5 和表 6 显示了变量和约束的概述。

节点变量。为了表示每个计算节点上的模型放置，我们在 MILP 公式中引入了两组变量。假设模型总共有 L 层，并且每个计算节点保存模型的连续子集。对于每个计算节点 c_i，我们引入一个整数变量 s_i 来表示第一层 c_i。假设计算节点 c_i 在其 GPU 上最多可以容纳 k 层，我们进一步引入 k 二进制变量 $b_i^1, b_i^2, ..., b_i^k$ 来指示节点 c_i 容纳的层数（如果 c_i 容纳 j 层，则 $b_i^j = 1$）。我们选择使用 k 二元变量（而不是层数的一个整数）来表达模型放置，因为这一公式有助于表达推理吞吐量约束，如下所述。这 c_i 的结束层索引可以表示为 $e_i = s_i + \sum_{j=1}^{k} j \cdot b_i^j$。因此，$c_i$ 保留范围 $[s_i, e_i)$ 中的层。

连接变量。我们引入两组变量来限制可以通过每个网络连接的推理请求的数量。对于计算节点c_i和c_j之间的网络连接，我们引入一个实变量$f_{i,j}$来表示图抽象中从c_i^{out}到c_j^{in}的流量。我们进一步引入一个二进制变量 $d_{i,j}$ 来表示网络连接是否有效（如 4.3 节中定义）。我们下面介绍的约束将使用 $d_{i,j}$ 来确保请求只能通过有效的连接传输。对于协调器节点和计算节点之间的网络连接，我们同样引入与上面类似的两个变量，但将 i/j 替换为 source/sink。

约束1：模型放置。为了确保 MILP 求解器找到的模型放置有效，我们需要为每个计算节点 c_i 设置以下两个约束。首先，c_i 应该只有一个有效的模型放置，即 $\sum_{j=1}^{k} b_i^j = 1$。此外，第一层和最后一层 c_i 必须在 L 层的范围内，即 $0 \le s_i < L$ 和 $e_i \le L$。 （c_i 保存范围 $[s_i, e_i)$ 内的层）

约束2：流量守恒。对于每个计算节点 c_i，由于流量守恒，进入 c_i^{in} 的流量总和必须等于流出 c_i^{out} 的流量总和。该约束可以表示为 $\sum_u f_{u,i} = \sum_v f_{i,v}$，其中 u 和 v 枚举了除 i 之外的所有节点。

约束 3：推理吞吐量。对于计算节点 c_i，经过 (c_i^{in}, c_i^{out}) 的流量不应超过其最大推理吞吐量。我们可以用 $\sum_u f_{ui} \le \sum_{j=1}^{k} b_i^j \cdot T_j$ 施加这个约束。这里，T_j是一个常量，表示节点c_i在持有j层时一秒钟可以处理的最大令牌数量，这是通过一次性分析过程获得的。

约束4：连接有效性。我们需要判断网络连接的有效性，以了解请求是否可以通过它们传输。对于从协调器节点到计算节点 c_i 的网络连接，仅当 c_i 保存模型的第一层时才有效。为了用 MILP 表达这个约束，我们需要将其线性化为以下形式：$s_i \le L \cdot (1 - d_{source,i})$。类似地，对于从计算节点 c_i 到协调器的网络连接，我们用 $L \cdot d_{i,sink} \le e_i$ 约束其有效性。对于计算节点 c_i 到 c_j 的网络连接，其有效性 $d_{i,j}$ 由 $s_j \le e_i < e_j$ 是否成立来确定。为了线性化这个条件，我们需要引入两个二元辅助变量 $cond_{i,j}^1$ 和 $cond_{i,j}^2$。仅当 $s_j \le e_i$ 时 $cond_{i,j}^1$ 才取值 1，可以线性化为 $(L+1)(1 - cond_{i,j}^1) \ge s_j - e_i$。仅当 $e_i < e_j$ 时 $cond_{i,j}^2$ 才取值 1，可以线性化为 $e_j - e_i \ge 1 - (L+1)(1 - cond_{i,j}^2)$。仅当两个二进制辅助变量都为 true 时，网络连接才有效，可以表示为 $d_{i,j} \le 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$。

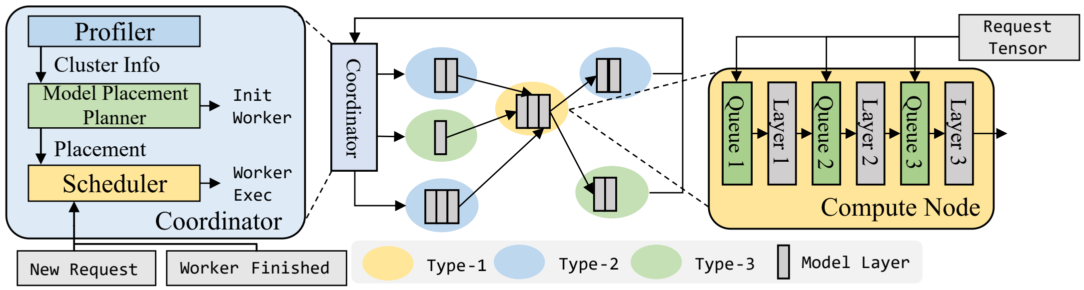


Figure 3. Helix overview. In Helix, the coordinator plans model placement as described in Sec. 4.4. We only need to run model placement once for each cluster. When a new request arrives, the coordinator node runs Helix scheduler to assign it a per-request pipeline and sends it to the first node in the pipeline. Each compute node in the pipeline performs inference on the request on the layers it is responsible for and sends the (output for the) request to the next node in the pipeline. When the last node in the pipeline finishes performing inference on its layers, it will send the output token for the request to the coordinator (Worker Finished). The coordinator schedules generation of the next token for the request using the same pipeline.

We remark that if $s_j < e_i < e_j$, then requests coming from c_i will only infer layers $[e_i, e_j)$ on c_j. We call this *partial inference*. If partial inference is not allowed, then the connection validity constraints can be simplified to $d_{i,j} = 1$ only if $e_i = s_j$, which linearizes to two constraints $L \cdot d_{i,j} \leq L + s_j - e_i$ and $L \cdot d_{i,j} \leq L - s_j + e_i$.
Constraint-5: transmission throughput. We only allow flow to pass through valid network connections, and the flow should not be larger than the connection's maximum transmission throughput. To enforce this constraint, we add $f_{i,j} \leq d_{i,j} \cdot S_{i,j}$ as a constraint into the MILP problem. $S_{i,j}$ is the maximum number of tokens that can be transmitted through the network connection, which can be calculated via profiling and using methods mentioned in Sec. 4.3.
Optimization target. The MILP problem aims to find a model placement that satisfies all constraints and yields the highest max flow for the cluster. This optimization target can be expressed as maximizing the sum of flow from source, i.e. maximizing $\sum_i f_{source,i}$.
MILP solution orchestration. After the MILP solver finds a solution that satisfies all constraints, we can orchestrate it into a model placement plan and construct the graph abstraction of the cluster. For compute node c_i, s_i and e_i give us the model layers c_i should load into its GPU.

4.5 Analyzing and Speeding up MILP

As Table 5 and 6 show, the number of variables and constraints in the MILP problem scales linearly with the number of compute nodes and network connections. For large clusters with more than 40 nodes, it may still take hours before the MILP solver gives a reasonably good solution. To expedite the MILP solving process for large clusters, we introduce three optimizations. First, we prune some of the slow network connections in the cluster. Evaluation in Sec. 6.8 shows that this effectively reduces the problem size without sacrificing much performance. Second, we hint the MILP solver with solutions found by heuristic methods. Since the problem has an exponential solution space, the MILP solver can only cover a small portion within a limited solving time budget. Using solutions from heuristic methods as starting points for the MILP problem expedites the optimization process, especially for large clusters. Sec. 6.8 shows the necessity of starting from heuristic solutions for large clusters. Finally, we notice that the max serving throughput of a cluster is always bounded by the sum of compute throughput of all compute nodes averaged by the total number of layers. The MILP solver uses this as an early stop criterion and stops when it finds a solution that is very close to this upper bound. We remark that, for further scaling of Helix to hundreds or even thousands of nodes, one viable approach is to first partition the nodes into multiple smaller clusters using heuristics and then apply Helix independently.

4.6 Replacing MILP with LP or Heuristics?

A common approach for speeding up MILP problems is to relax them to a linear program (LP) by relaxing the integer variables to be linear variables and obtaining a valid solution to the original problem via methods such as rounding the resulting linear variables. We remark that this approach is not viable for the MILP problem above. This is because the resulting solution from the LP cannot be easily converted to a valid solution of the original problem. The variables for model placement (s_i and b_i^j) decide the edge validity variables $d_{i,j}$, which in turn decides the flow variables $f_{i,j}$. Rounding the non-integral values of model placement variables in the relaxed solution may invalidate some or all network connections and thus drastically changing the max flow.

Our preliminary exploration also indicates that using simpler heuristics cannot guarantee good performance across various cluster setups, because of the exponential solution space of this problem. Sec. 6.6 shows that the model placement found by Helix with MILP is much better than that of the heuristic baselines.

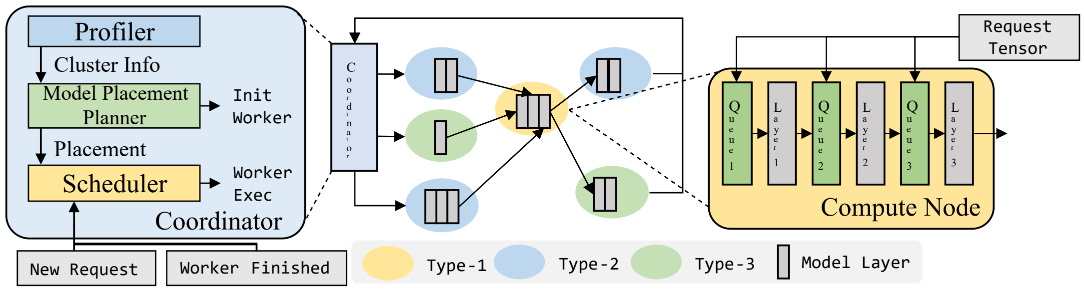


图 3. 螺旋概述。在 Helix 中，协调员按照第 2 节中所述计划模型放置。 4.4.我们只需要为每个集群运行一次模型放置。当新请求到达时，协调器节点运行 Helix 调度程序为其分配每个请求管道并将其发送到管道中的第一个节点。管道中的每个计算节点对其负责的层上的请求执行推理，并将请求（的输出）发送到管道中的下一个节点。当管道中的最后一个节点完成对其各层的推理时，它将向协调器发送请求的输出令牌（Worker Finished）。协调器使用相同的管道安排请求的下一个令牌的生成。

我们注意到，如果 $s_j < e_i < e_j$ ，那么来自 c_i 的请求将仅推断 c_j 上的层 $[e_i, e_j)$ 。我们称之为部分推理。如果不允许部分推理，则仅当 $e_i = s_j$ 时，连接有效性约束才能简化为 $d_{i,j} = 1$ ，这线性化为两个约束 $L \cdot d_{i,j} \leq L + s_j - e_i$ 和 $L \cdot d_{i,j} \leq L - s_j + e_i$ 。

约束5：传输吞吐量。我们只允许流量通过有效的网络连接，并且流量不应大于连接的最大传输吞吐量。为了强制执行此约束，我们将 $f_{i,j} \leq d_{i,j} \cdot S_{i,j}$ 作为约束添加到 MILP 问题中。 $S_{i,j}$ 是可以通过网络连接传输的最大令牌数量，可以通过分析并使用第 2 节中提到的方法来计算。 4.3.

优化目标。 MILP 问题旨在找到满足所有约束并为集群产生最高最大流量的模型放置。该优化目标可以表示为最大化来自源的流量之和，即最大化 $\sum_i f_{source,i}$ 。

MILP 解决方案编排。 MILP求解器找到满足所有约束的解决方案后，我们可以将其编排成模型放置计划并构建集群的图抽象。对于计算节点 c_i 、 s_i 和 e_i 为我们提供了应加载到其 GPU 中的模型层 c_i。

4.5 分析和加速MILP

如表 5 和表 6 所示，MILP 问题中的变量和约束的数量与计算节点和网络连接的数量呈线性关系。对于超过 40 个节点的大型集群，MILP 求解器可能仍需要几个小时才能给出相当好的解决方案。为了加快大型集群的 MILP 求解过程，我们引入了三种优化。首先，我们删除集群中一些慢速的网络连接。秒中的评估。 6.8 表明，这有效地减少了问题的规模，而不会牺牲太多的性能。其次，我们用启发式方法找到的解决方案来提示 MILP 求解器。自从出现问题由于 MILP 求解器具有指数解空间，因此只能在有限的求解时间预算内覆盖一小部分。使用启发式方法的解决方案作为 MILP 问题的起点可以加快优化过程，尤其是对于大型集群。秒。 6.8 显示了从大型集群的启发式解决方案开始的必要性。最后，我们注意到集群的最大服务吞吐量始终受所有计算节点的计算吞吐量之和（除总层数的平均值）的限制。 MILP 求解器使用此作为早期停止标准，并在找到非常接近此上限的解时停止。我们注意到，为了将 Helix 进一步扩展到数百甚至数千个节点，一种可行的方法是首先使用启发式方法将节点划分为多个较小的集群，然后独立应用 Helix。

4.6 用LP或启发式代替MILP?

加速 MILP 问题的常见方法是将整数变量放松为线性变量，并通过舍入所得线性变量等方法获得原始问题的有效解，从而将其放松为线性程序 (LP)。我们注意到这种方法对于上面的 MILP 问题是不可行的。这是因为 LP 得到的解决方案无法轻松转换为原始问题的有效解决方案。模型放置的变量（s_i 和 b_i^j ）决定边有效性变量 $d_{i,j}$ ，而边有效性变量又决定流变量 $f_{i,j}$ 。在宽松的解决方案中对模型放置变量的非整数值进行四舍五入可能会使部分或全部网络连接无效，从而极大地改变最大流量。

我们的初步探索还表明，由于这个问题的指数解空间，使用更简单的启发式方法不能保证在各种集群设置中都有良好的性能。秒。 6.6 显示 Helix 与 MILP 发现的模型放置比启发式基线的模型放置要好得多。

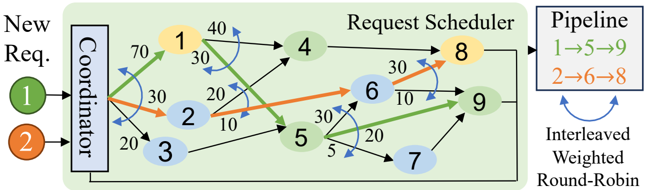
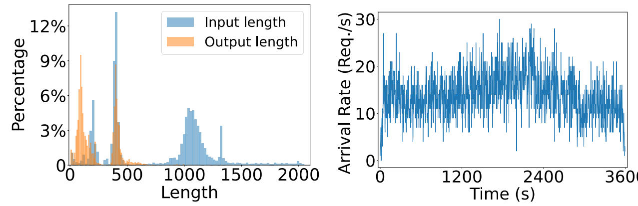


Figure 4. Topology graph of a cluster, where each vertex is a compute node, and each edge is a valid network connection. Numbers over edges represent the flow over the network connection in the max flow solution. The pipelines used to schedule requests 1 and 2 are shown on the right.



(a) Length distribution. **(b)** Arrival rate.

Figure 5. Statistics of Azure Conversation dataset.

5 Helix Runtime

This section discusses the runtime scheduling of requests in Helix. When the coordinator node receives a new request, it runs Helix's request scheduler to assign the request a *per-request pipeline*, which we will introduce in Sec. 5.1. Then the coordinator node sends the request to the first compute node in the pipeline. When a compute node receives a requests, it performs inference on the request using the layers it is responsible for in that pipeline and sends the request to the following compute node. Fig. 3 shows the overview of Helix.

5.1 Scheduler Design: Per-Request Pipelines

To infer a request in the cluster, the scheduler needs to assign a *pipeline* for the request. The pipeline contains a sequence of stages, where each stage specifies a compute node and the layers to infer on the compute node. A valid pipeline must infer each layer of the model exactly-once and in correct order when running the stages sequentially. Existing works [5, 19] use fixed pipelines, in which each pipeline contains a disjoint set of machines, and assign requests to those pipelines. In Helix, instead of using fixed-pipelines, we propose a *per-request pipeline* assignment approach, wherein each request will have its own pipeline and the pipelines may intersect with each other. The total number of possible pipelines equals the number of possible paths from source to sink in the graph abstraction of the cluster. The abundant number of pipelines allows the scheduling to better fit the capacity of the compute nodes and network connections. Our Max-Flow formulation enables us to create the per-request pipelines.

The Helix request scheduler performs scheduling based on the cluster's *topology graph* (Fig. 4). In the topology graph, vertices correspond to the nodes in the cluster. Directed edges correspond to the valid network connections (under the model placement found by solving the MILP in Sec. 4). We bind an interleaved weighted round-robin (IWRR) [51] scheduler to each vertex. The IWRR scheduler takes as input a list of candidates and their weights. To schedule a request, it selects a candidate with frequency proportional to its weight. For each vertex u, the IWRR scheduler's candidates contain all vertices v such that directed edge (u, v) exists (i.e. (u, v) represents a valid network connection). The weight of v equals the flow over the network connection of (u, v) in the max flow solution. Using IWRR allows us to schedule requests following the max flow without creating bursts.

Helix's request scheduler runs on the coordinator node. When a new request arrives, it first uses the IWRR scheduler of the vertex representing the coordinator node to determine the compute node c_1 for the first pipeline stage. Then, it uses the IWRR scheduler of the vertex representing c_1 to determine the compute node c_2 for the second stage, and repeats this process until a valid pipeline is established. Fig. 4 shows the scheduling of two requests. After setting the request's pipeline, the coordinator node sends the request to the first compute node in the pipeline to begin inference. During inference, Helix adopts a *dynamic batching* strategy where each node includes all requests received during the processing of the previous batch to form a new one. This best-effort batching occurs without additional waiting periods.

5.2 KV-Cache Estimation

When serving LLMs, each GPU has a limited amount of VRAM to store the KV-cache of requests during inference. If the requests running concurrently on the GPU require more KV-cache than this limit, the execution engine has to offload some requests to main memory, which significantly harms throughput. However, we do not know exactly how much KV-cache each request will use because the length of the output is unknown before inference finishes. Therefore, in the scheduler we maintain an estimation of KV-cache usage of all compute nodes using average output length, and mask out compute nodes that exceed the high water mark when running IWRR. We can schedule more requests to the compute nodes only after some requests currently running on those nodes have finished. This mask ensures that we do not oversubscribe the GPU's KV-cache.

6 Evaluation

In this section, we aim to answer the following questions.

- Can Helix provide higher throughput for heterogeneous GPUs in a single cluster? (Sec. 6.3)
- Does Helix sacrifice latency for throughput? (Sec. 6.3)
- Can Helix provide higher throughput for heterogeneous GPUs in geo-distributed clusters? (Sec. 6.4)

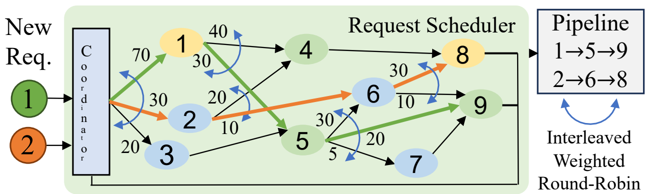


图 4. 集群的拓扑图，其中每个顶点都是一个计算节点，每条边都是一个有效的网络连接。边上的数字表示最大流量解决方案中网络连接上的流量。用于调度请求 1 和 2 的管道如右侧所示。

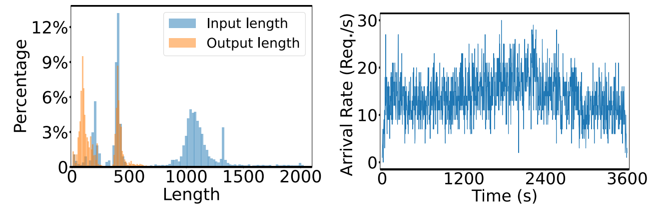
5 螺旋运行时

本节讨论 Helix 中请求的运行时调度。当协调器节点收到新请求时，它会运行 Helix 的请求调度程序，为该请求分配一个每请求管道，我们将在第 2 节中介绍。 5.1. 然后协调器节点将请求发送到管道中的第一个计算节点。当计算节点收到请求时，它会使用它在该管道中负责的层对请求进行推理，并将请求发送到后续计算节点。图 3 显示了 Helix 的概述。

5.1 调度程序设计：每个请求的管道

为了推断集群中的请求，调度程序需要为该请求分配管道。该管道包含一系列阶段，其中每个阶段指定一个计算节点以及在计算节点上推断的层。当顺序运行各阶段时，有效的管道必须准确地推断模型的每一层一次并按正确的顺序。现有的工作 [5, 19] 使用固定管道，其中每个管道包含一组不相交的机器，并将请求分配给这些管道。在 Helix 中，我们没有使用固定管道，而是提出了一种按请求管道分配方法，其中每个请求都有自己的管道，并且管道可以相互交叉。可能的管道总数等于集群的图形抽象中从源到汇的可能路径的数量。丰富的管道数量使得调度能够更好地适应计算节点和网络连接的容量。我们的最大流量公式使我们能够创建每个请求的管道。

Helix请求调度器根据集群的拓扑图进行调度（图4）。在拓扑图中，顶点对应于集群中的节点。有向边对应于有效的网络连接（在通过求解第 4 节中的 MILP 找到的模型放置下）。我们将交错加权循环（IWRR）[51]调度程序绑定到每个顶点。 IWRR 调度程序将候选列表及其权重作为输入。为了安排请求，它会选择一个与其权重成比例的频率的候选者。对于每个顶点 u，IWRR 调度程序的候选者包含所有顶点 v，使得有向边 (u, v) 存在（即 (u, v)



(a) Length distribution. **(b)** Arrival rate.

图 5.Azure 对话数据集的统计信息。

代表有效的网络连接）。 v的权重等于最大流量解中 (u, v)网络连接上的流量。使用 IWRR 允许我们按照最大流量安排请求，而不会产生突发。

Helix 的请求调度程序在协调器节点上运行。当新请求到达时，它首先使用代表协调器节点的顶点的 IWRR 调度程序来确定第一个管道阶段的计算节点 c_1。然后，它使用代表c_1的顶点的IWRR调度器来确定第二阶段的计算节点c_2，并重复此过程直到建立有效的管道。图 4 显示了两个请求的调度。设置请求的管道后，协调器节点将请求发送到管道中的第一个计算节点以开始推理。在推理过程中，Helix 采用动态批处理策略，其中每个节点包含前一批处理过程中收到的所有请求，以形成新的一批。这种尽力批处理的发生无需额外的等待期。

5.2 KV-缓存估计

当服务 LLM 时，每个 GPU 都有有限数量的 VRAM 来存储推理期间请求的 KV 缓存。如果在 GPU 上同时运行的请求需要的 KV 缓存超过此限制，则执行引擎必须将一些请求卸载到主内存，这会严重损害吞吐量。然而，我们并不确切知道每个请求将使用多少 KV 缓存，因为在推理完成之前输出的长度是未知的。因此，在调度程序中，我们使用平均输出长度来估计所有计算节点的 KV 缓存使用情况，并在运行 IWRR 时屏蔽掉超出高水位线的计算节点。仅当当前在这些节点上运行的某些请求完成后，我们才能向计算节点调度更多请求。这个掩码确保我们不会过度订阅 GPU 的 KV 缓存。

6 评价

在本节中，我们旨在回答以下问题。

- Helix 能否为单个集群中的异构 GPU 提供更高的吞吐量？ （第 6.3 节）
- Helix 是否会牺牲延迟来换取吞吐量？ （第 6.3 节）
- Helix 能否为地理分布式集群中的异构 GPU 提供更高的吞吐量？ （第 6.4 节）

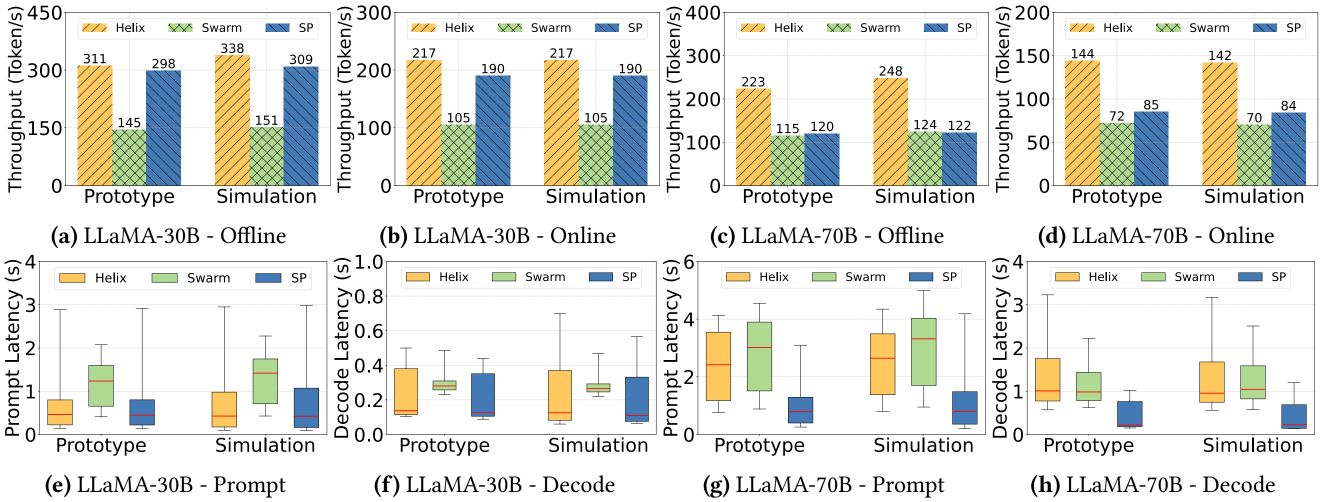


Figure 6. Single cluster results: (a - d) decode throughput, **(e - h)** prompt and decode latency for online serving. The box shows the 25 and 75 percentile. The whisker shows the 5 and 95 percentile. The red line shows median.

Table 7. Network bandwidth between machines in asia-east, us-central, europe-west and australia-southeast. Measured with iperf3 on Google Compute Engine.

Receiver	Sender			
	asia-east2-a	us-central1-f	eu-west3-c	au-se1-c
asia-east2-a	/	123 Mbps	67 Mbps	175 Mbps
us-central1-f	122 Mbps	/	204 Mbps	123 Mbps
eu-west3-c	61 Mbps	196 Mbps	/	54 Mbps
au-se1-c	159 Mbps	118 Mbps	63 Mbps	/

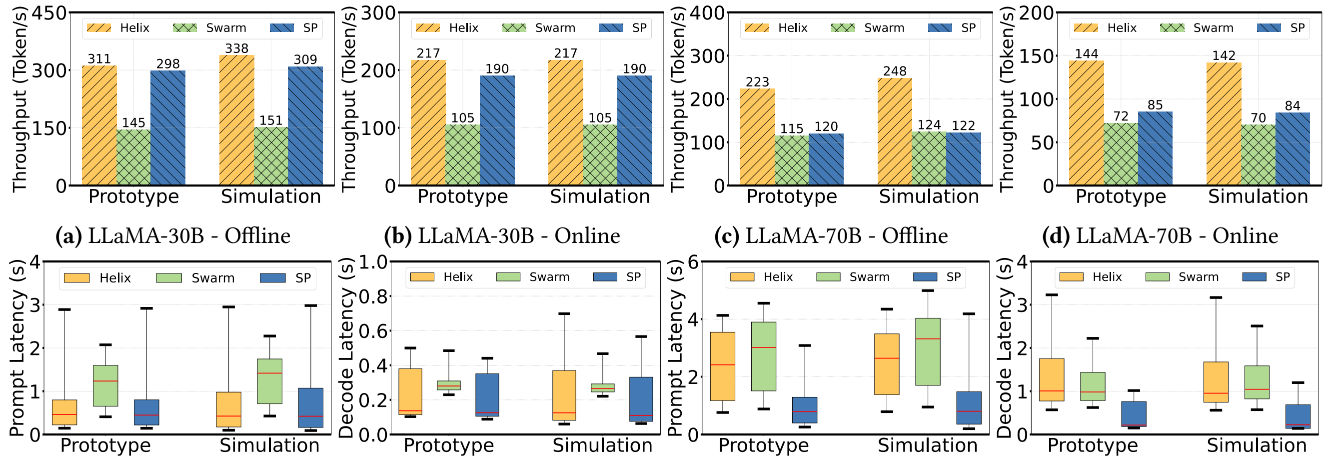
- Can Helix provide consistently high throughput when the degree of GPU heterogeneity increases? (Sec. 6.5)
- Why does Helix achieve better performance compared to existing systems (Sec. 6.6 and 6.7)?

6.1 Implementation

Helix prototype. We implemented a multi-replica pipeline parallel system with 1.5k LoC in Python and 1.7k LoC in C++. For model execution, we adopt the latest release of vLLM [22] (0.4.0post1) to avoid re-implementing basic LLM inference optimizations. We implemented a *unified page pool* atop vLLM to support partial inference. We use ZeroMQ [52] for inter-node communication and Gurobi [14] as MILP solver.
Simulator. We also implemented a simulator for distributed LLM inference with 14k LoC in Python. It supports simulation for heterogeneous GPUs and network conditions. It gives us the flexibility to explore more diverse settings for network, GPU heterogeneity and cluster scale. Sec. 6.3 evaluates the fidelity of the simulator and shows that the simulation errors are less than 5% for all metrics.

6.2 Experiment Setup

Models. We evaluate Helix on LLaMA [55, 56], a representative and popular open-source Transformer model family. Specifically, we use LLaMA-1 30B and LLaMA-2 70B to study the system performance on models of different sizes. We run model inference with half-precision (FP16). In subsequent sections, we refer to these two models as LLaMA 30B and LLaMA 70B.
Cluster setup. We evaluate with three cluster setups: (1) single cluster (Sec. 6.3), (2) geo-distributed clusters (Sec. 6.4), and (3) high GPU heterogeneity cluster (Sec. 6.5). First, the *single cluster* setup contains 4 A100 nodes, 8 L4 nodes and 12 T4 nodes connected by 10Gb/s network. We configure the network bandwidth to 10 Gb/s, as this rate is sufficient to ensure that LLM serving is limited by GPU throughput rather than network capacity. We allocate the nodes within one region on the Google Cloud. Second, the *geo-distributed clusters* contain three clusters with (i) 4 A100 nodes, (ii) 2 L4 nodes + 8 T4 nodes, and (iii) 6 L4 nodes + 4 T4 nodes. Inter-cluster communication has an average bandwidth of 100 Mb/s and an average latency of 50 ms. We configure the bandwidth to 100 Mb/s to simulate cross-region network limitations, based on our profiling results in Table 7. Finally, the *high GPU heterogeneity cluster* contains 4 A100 nodes, 6 V100 nodes, 8 L4 nodes, 10 T4 nodes, 4 2×L4 nodes, 6 2×T4 nodes and 4 4×T4 nodes. The latter two setups are performed in simulation. Our evaluation of the prototype system focuses on single-GPU nodes, as multi-GPU nodes are much more difficult to allocate in the cloud [53]. Our implementation also works for multi-GPU nodes by leveraging tensor model parallelism across GPUs on the same node as supported by vLLM [22].
Traces. The traces we use come from Azure Conversation dataset [40]. Fig. 5 shows the length distribution and arrival rate of this dataset. We remove requests with input lengths larger than 2048 or output lengths larger than 1024 to maintain reasonable runtime memory usage for vLLM.



(a) LLaMA-30B - Offline **(b)** LLaMA-30B - Online **(c)** LLaMA-70B - Offline **(d)** LLaMA-70B - Online

(e) LLaMA-30B - Prompt **(f)** LLaMA-30B - Decode **(g)** LLaMA-70B - Prompt **(h)** LLaMA-70B - Decode

图 6. 单集群结果：(a - d) 解码吞吐量，(e - h) 在线服务的提示和解码延迟。该框显示 25 和 75 百分位。胡须显示第 5 个和第 95 个百分位。红线显示中位数。

表 7. 亚洲东部、美国中部、欧洲西部和澳大利亚东南部机器之间的网络带宽。在 Google Compute Engine 上使用 iperf3 进行测量。

Receiver	Sender			
	asia-east2-a	us-central1-f	eu-west3-c	au-se1-c
asia-east2-a	/	123 Mbps	67 Mbps	175 Mbps
us-central1-f	122 Mbps	/	204 Mbps	123 Mbps
eu-west3-c	61 Mbps	196 Mbps	/	54 Mbps
au-se1-c	159 Mbps	118 Mbps	63 Mbps	/

- 当 GPU 异构程度增加时，Helix 能否提供持续的高吞吐量？ （第 6.5 节）
- 与现有系统相比，为什么 Helix 能够实现更好的性能（第 6.6 节和 6.7 节）？

6.1 实施

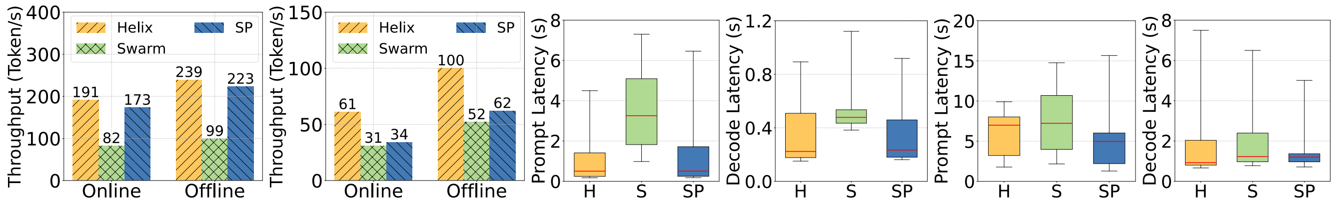
螺旋原型。我们在 Python 中实现了一个具有 1.5k LoC 的多副本管道并行系统，在 C++ 中实现了具有 1.7k LoC 的多副本管道并行系统。对于模型执行，我们采用最新版本的 vLLM [22] (0.4.0post1) 以避免重新实现基本的 LLM 推理优化。我们在 vLLM 之上实现了统一的页面池来支持部分推理。我们使用 ZeroMQ [52] 进行节点间通信，使用 Gurobi [14] 作为 MILP 求解器。模拟器。我们还在 Python 中实现了一个具有 14k LoC 的分布式 LLM 推理模拟器。它支持异构 GPU 和网络条件的模拟。它使我们能够灵活地探索更多样化的网络、GPU 异构性和集群规模设置。秒。 6.3 评估了模拟器的保真度，结果显示所有指标的模拟误差均小于 5%。

6.2 实验设置

模型。我们在 LLaMA [55, 56] 上评估 Helix，LLaMA 是一个具有代表性且流行的开源 Transformer 模型系列。具体来说，我们使用LLaMA-1 30B和LLaMA-2 70B来研究不同尺寸模型上的系统性能。我们以半精度 (FP16) 运行模型推理。在后续部分中，我们将这两个型号称为 LLaMA 30B 和 LLaMA 70B。

集群设置。我们使用三种集群设置进行评估：(1) 单集群（第 6.3 节）、(2) 地理分布式集群（第 6.4 节）和 (3) 高 GPU 异构性集群（第 6.5 节）。首先，单个集群设置包含通过 10Gb/s 网络连接的 4 个 A100 节点、8 个 L4 节点和 12 个 T4 节点。我们将网络带宽配置为 10 Gb/s ，因为该速率足以确保 LLM 服务受到 GPU 吞吐量而不是网络容量的限制。我们在 Google Cloud 上的一个区域内分配节点。其次，地理分布式集群包含三个集群，其中（i）4 个 A100 节点，（ii）2 个 L4 节点+ 8 个 T4 节点，以及（iii）6 个 L4 节点+ 4 个 T4 节点。集群间通信的平均带宽为100Mb/s，平均延迟为50ms。根据表 7 中的分析结果，我们将带宽配置为 100 Mb/s 来模拟跨区域网络限制。最后，高 GPU 异构集群包含 4 个 A100 节点、6 个 V100 节点、8 个 L4 节点、10 个 T4 节点、4 个 2×L4 节点、6 个 2×T4 节点和 4 个 4×T4 节点。后两种设置是在模拟中执行的。我们对原型系统的评估侧重于单 GPU 节点，因为多 GPU 节点在云中分配起来要困难得多 [53]。我们的实现也适用于多 GPU 节点，通过利用 vLLM [22] 支持的同一节点上 GPU 之间的张量模型并行性。

痕迹。我们使用的跟踪来自 Azure 对话数据集 [40]。图 5 显示了该数据集的长度分布和到达率。我们删除输入长度大于 2048 或输出长度大于 1024 的请求，以维持 vLLM 合理的运行时内存使用量。



(a) LLaMA-30B (b) LLaMA-70B (c) LLaMA-30B (d) LLaMA-30B (e) LLaMA-70B (f) LLaMA-70B

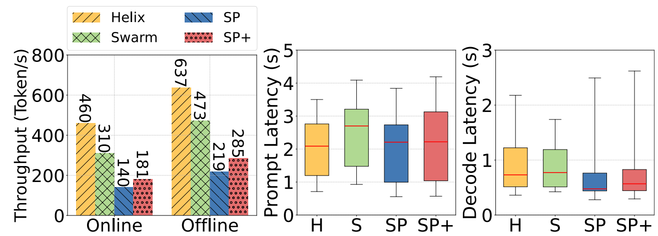
Figure 7. Geo-distributed clusters: (a - b) decode throughput, **(c - f)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm. Box: 25 and 75 percentile. Whisker: 5 and 95 percentile. Red line: median.

The pruned dataset contains 16657 requests with an average input length of 763 and an average output length of 232. We remark that this dataset is more challenging than the ones used by prior work [22] due to longer request length. We have two settings for arrival rates. For **online setting**, we use real arrival rates from Azure Conversation dataset. We scale the average arrival rate to 75% of the cluster's peak throughput to avoid bursts of requests leading to OOM in the system. For **offline setting**, we allow requests to arrive at the rate needed to fully utilize the cluster. This mimics running offline inference on a dataset. We refer to the two settings as *online* and *offline serving*.

Experiment duration. For online setting, we warm up the cluster for 30s and test for 30 minutes. For offline setting, we warm up the cluster for 1 minute and test for 10 minutes. This amount of time is sufficient for our evaluations and we do not run longer to reduce unnecessary experimental cost.

Helix setup. We allow the MILP solver to search w/ and w/o partial inference and cluster pruning. We also hint the MILP solver with solutions from Petals / Swarm / separate pipelines. Solving times out when the MILP solver does not find better solutions in 10 minutes. The max search budget for each cluster setup is 4 hours on a 14-core CPU.

Baselines. To the best of our knowledge, there are no heterogeneous LLM serving systems with model placement and scheduling applicable to our settings. Therefore, we adopt ideas from a heterogeneous LLM training system, Swarm [45], to build a competitive heterogeneous baseline. We also build another baseline that handles GPU heterogeneity by serving one model replica with each type of machine. We refer to the two baselines as Swarm and separate pipelines (SP). To ensure a fair comparison, we implemented the SP baseline within our system rather than utilizing other systems. We further compare with the model placement of Petals [5] in Sec. 6.6, which is a decentralized LLM serving system for volunteer computing. We do not compare with Petals in end-to-end serving as it lacks centralized request scheduling. For Swarm, we implemented their model placement and scheduling algorithm in our system, since their original system can not be used for inference. We set the number of pipeline stages to the minimum that allows the weakest GPU to hold one stage with half its VRAM. This minimizes the pipeline depth and leaves enough VRAM for KV-cache, both of which are crucial to performance.



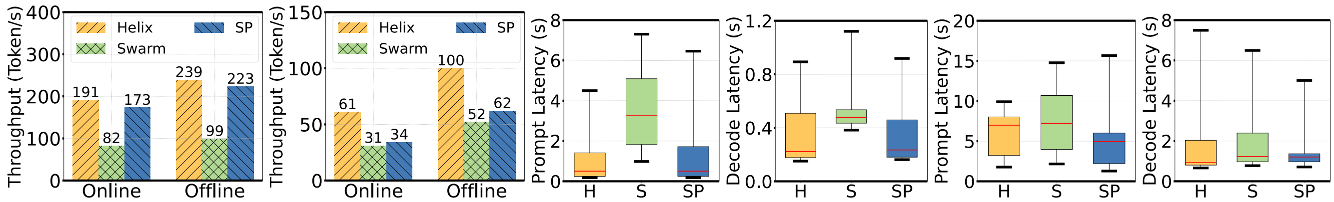
(a) LLaMA-70B (b) LLaMA-70B (c) LLaMA-70B

Figure 8. High GPU-heterogeneity clusters setup: (a) decode throughput, **(b - c)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm.

Metrics. For offline serving, we report average *decode throughput*, which is the number of tokens generated per second. For online serving, we further report average *prompt latency* and *decode latency*, which is the average latency for parsing user input and generating new tokens, respectively.

6.3 Single Cluster

This section evaluates Helix with online and offline serving of LLaMA 30B and 70B in the single cluster setup. Fig. 6 shows the results. For LLaMA 30B, each GPU type has enough nodes to serve at least one individual pipeline. The best model placement discovered by Helix serves three replicas of the model, each with one type of GPU. As a result, Helix and SP achieve similar performance: Helix has 4% and 14% higher decode throughput for offline and online serving because of its better KV-cache utilization; the prompt latency is 2% lower and decode latency is 10% higher. Compared with Swarm, Helix achieves 2.14× and 2.07× higher decode throughput for offline and online serving. Helix also reduces prompt and decode latency by 32% and 12% for online serving. We find that Swarm's model placement introduces a bottleneck and under-utilizes the A100 nodes, which we discuss in more detail in Sec. 6.6. We note that the comparatively higher latency observed in our experiments, relative to other LLM serving systems [22, 40], can be attributed to our use of less powerful GPUs (T4 and L4) in contrast to the A100 and H100 GPUs utilized in other studies.



(a) LLaMA-30B **(b)** LLaMA-70B **(c)** LLaMA-30B **(d)** LLaMA-30B **(e)** LLaMA-70B **(f)** LLaMA-70B

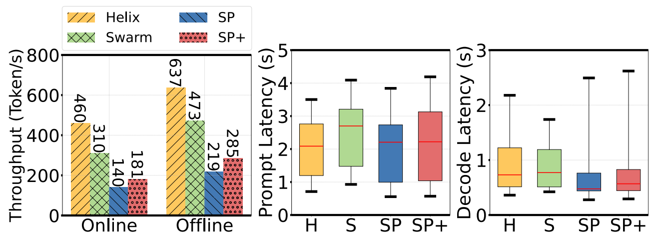
图 7. 地理分布式集群：(a - b) 解码吞吐量，(c - f) 在线服务的提示和解码延迟。 “H”代表Helix， “S”代表Swarm
。方框：25 和 75 百分位。晶须：5% 和 95%。红线：中位数。

修剪后的数据集包含 16657 个请求，平均输入长度为 763，平均输出长度为 232。我们注意到，由于请求长度较长，该数据集比先前工作 [22] 使用的数据集更具挑战性。我们有两种到达率设置。对于在线设置，我们使用 Azure 对话数据集的实际到达率。我们将平均到达率扩展到集群峰值吞吐量的 75%，以避免突发请求导致系统 OOM。对于离线设置，我们允许请求达到充分利用集群所需的速率。这模仿了在数据集上运行离线推理。我们将这两种设置称为在线服务和离线服务。

实验持续时间。对于在线设置，我们将集群预热 30 秒并测试 30 分钟。对于离线设置，我们预热集群 1 分钟并测试 10 分钟。这个时间对于我们的评估来说是足够的，我们不会跑更长的时间来减少不必要的实验成本。

螺旋设置。我们允许 MILP 求解器使用和不使用部分推理和聚类修剪进行搜索。我们还使用 Petals/Swarm/单独管道的解决方案提示 MILP 求解器。当 MILP 求解器在 10 分钟内找不到更好的解时，求解就会超时。每个集群设置的最大搜索预算在 14 核 CPU 上为 4 小时。

基线。据我们所知，不存在适用于我们设置的模型放置和调度的异构 LLM 服务系统。因此，我们采用异构 LLM 培训系统 Swarm [45] 的思想来构建有竞争力的异构基线。我们还构建了另一个基线，通过为每种类型的机器提供一个模型副本来处理 GPU 异构性。我们将这两个基线称为 Swarm 和分离管道 (SP)。为了确保公平比较，我们在我们的系统中实施了 SP 基线，而不是利用其他系统。我们进一步与第 2 节中 Petals [5] 的模型放置进行比较。 6.6，这是一个用于志愿者计算的去中心化LLM服务系统。我们不会在端到端服务方面与 Petals 进行比较，因为它缺乏集中的请求调度。对于Swarm，我们在我们的系统中实现了他们的模型放置和调度算法，因为他们的原始系统不能用于推理。我们将管道级数设置为最小值，允许最弱的 GPU 用一半的 VRAM 来容纳一个级。这



(a) LLaMA-70B **(b)** LLaMA-70B **(c)** LLaMA-70B

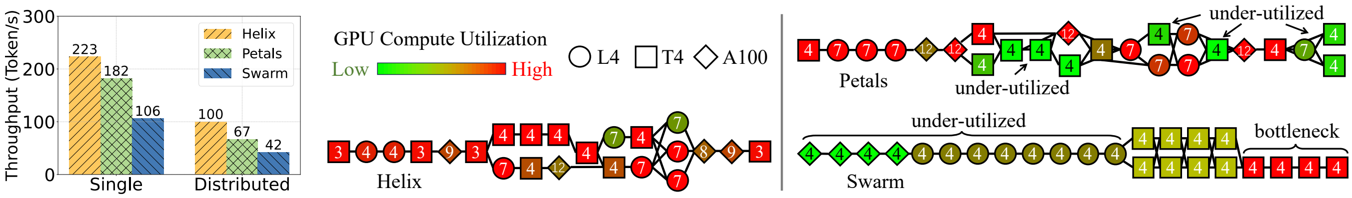
图 8. 高 GPU 异构性集群设置：(a) 解码吞吐量，(b - c) 在线服务的提示和解码延迟。 “H”代表Helix， “S”代表Swarm。

最大限度地减少管道深度并为 KV 缓存留下足够的 VRAM，这两者对于性能都至关重要。

指标。对于离线服务，我们报告平均解码吞吐量，即每秒生成的令牌数量。对于在线服务，我们进一步报告平均提示延迟和解码延迟，分别是解析用户输入和生成新令牌的平均延迟。

6.3 单集群

本部分通过在单集群设置中提供 LLaMA 30B 和 70B 在线和离线服务来评估 Helix。结果如图 6 所示。对于 LLaMA 30B，每种 GPU 类型都有足够的节点来服务至少一个单独的管道。 Helix 发现的最佳模型放置服务于模型的三个副本，每个副本都具有一种类型的 GPU。因此，Helix 和 SP 实现了相似的性能：由于其更好的 KV 缓存利用率，Helix 的离线和在线服务解码吞吐量分别高出 4% 和 14%；提示延迟降低 2%，解码延迟提高 10%。与 Swarm 相比，Helix 的离线和在线服务解码吞吐量分别提高了 2.14× 和 2.07×。 Helix 还将在线服务的提示和解码延迟分别减少了 32% 和 12%。我们发现 Swarm 的模型放置引入了瓶颈并且未充分利用 A100 节点，我们将在第 2 节中更详细地讨论这一点。 6.6.我们注意到，与其他 LLM 服务系统 [22, 40] 相比，我们的实验中观察到的延迟相对较高，这可归因于我们使用功能较弱的 GPU（T4 和 L4），而其他研究中使用的是 A100 和 H100 GPU。



(a) Decode throughput. **(b)** Model placement case study. Numbers on each node represent the number of layers the node holds.

Figure 9. (a) Comparing different model placement methods with offline serving of LLaMA 70B. **(b)** The case study shows model placements for serving LLaMA 70B on the single cluster setup.

For LLaMA 70B, nodes from a single GPU type can not serve one model replica by themselves while leaving enough VRAM for KV-cache. In this case, SP's throughput significantly decreases. Compared with SP, Helix achieves 1.86× and 1.69× higher decode throughput in offline and online serving. The average latency of Helix is higher than SP because SP serves majority of requests with A100 nodes and under-utilizes the L4 and T4 nodes. Compared with Swarm, Helix achieves 1.94× and 2.00× higher decode throughput in offline and online serving. Helix's prompt latency is 15% lower, but decode latency is 16% higher because of Helix's high GPU utilization. Similar to LLaMA 30B, Swarm under-utilizes A100 nodes because of its model placement.

We also conduct these experiments in the simulator (see Sec. 6.1) to examine its fidelity. The simulation results are shown alongside the real system evaluation results in Fig. 6. The average error of decode throughput is lower than 5%. The average error of prompt and decode latency is lower than 5% and 4% respectively. This indicates that our simulator achieves high fidelity and serves the purpose of comparing the performance of different methods.

6.4 Geo-Distributed Clusters

This section evaluates Helix for both online and offline serving of LLaMA 30B and 70B in geo-distributed clusters using our high-fidelity simulator in Sec. 6.1. Fig. 7 shows the results. Although this setup involves the same set of GPUs as the single cluster, the slow inter-cluster network causes all methods to have lower throughput and higher latency.

For LLaMA 30B, the best model placement found by Helix still consists of three pipelines served by three types of machines separately. Compared with SP, Helix has 7% and 10% higher decode throughput for offline and online serving. The prompt latency is 14% lower and decode latency is 2% higher. Compared with Swarm, Helix achieves 2.41× and 2.33× higher decode throughput for both online and offline serving. Helix also reduces prompt and decode latency by 66% and 24% for online serving.

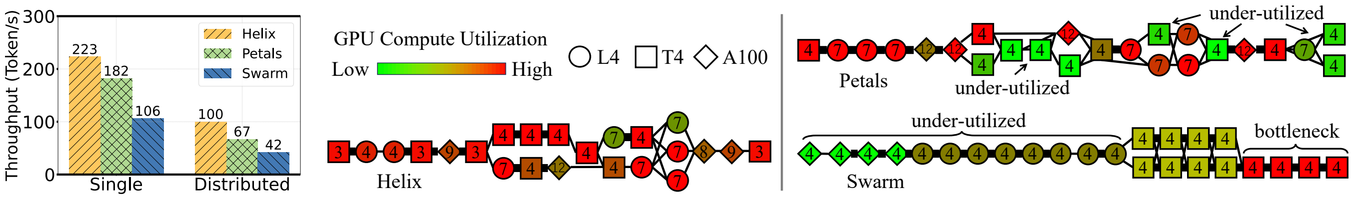
On the other hand, serving LLaMA 70B is more sensitive to slow network because of larger activation size and model depth. Helix reduces network overhead by using model placements with shallower pipeline depth. The model placement found by Helix reduces pipeline depth by 28% compared to Swarm. It is also 19% shallower than the one found by Helix when network is fast. Helix's model placement planner balances network overhead with single node's GPU utilization, achieving high throughput and low latency in geo-distributed GPU clusters. Compared with SP, Helix achieves 1.61× and 1.79× higher decode throughput for offline and online serving, respectively. SP has lower average latency because it serves most requests with fast GPUs, while 12 T4 nodes are under-utilized because of VRAM constraints. Compared with Swarm, Helix achieves 1.92× and 1.97× higher decode throughput for offline and online serving. Helix also reduces prompt and decode latency by 21% and 7%. We observe severe congestion during offline serving with Swarm. The average prompt latency reaches 71s, which is 7.5× that of Helix's. We will discuss more about the congestion of Swarm with a case study in Sec. 6.7.

6.5 High GPU-Heterogeneity Cluster

This section evaluates Helix on a highly heterogeneous GPU cluster with 42 compute nodes and 7 types of GPUs. Fig. 8 shows the results. In this cluster, V100, T4, and T4×2 nodes cannot form serving pipelines by themselves. We report the throughput without those machines for SP. We also try to build a mixed pipeline using those machines and report the number with the mixed pipeline as SP+. Compared with Swarm, SP and SP+, Helix achieves 1.37×, 2.91×, and 2.24× throughput for offline serving, and 1.48×, 3.29× and 2.54× for online serving. Helix also reduces prompt latency by 17%, 1% and 7% respectively. The average decode latency of Helix is slightly higher, because the baselines under-utilize the slow GPUs and serve most requests with fast GPUs with large VRAM. This result indicates that Helix can achieve consistently high performance when there are many types of GPUs in the cluster.

6.6 Model Placement Deep Dive

In this section, we analyze the impact of different model placement methods on the serving throughput of the cluster. We evaluate Helix's offline serving performance on both single and geo-distributed clusters, and compare Helix's model placement method with those used in Swarm and Petals.



(a) Decode throughput. **(b)** Model placement case study. Numbers on each node represent the number of layers the node holds.

图 9.(a) 将不同模型放置方法与 LLaMA 70B 离线服务进行比较。 (b) 案例研究显示了在单集群设置上为 LLaMA 70B 提供服务的模型布局。

对于 LLaMA 70B，来自单一 GPU 类型的节点无法单独为一个模型副本提供服务，同时为 KV 缓存留下足够的 VRAM。在这种情况下，SP 的吞吐量显着下降。与 SP 相比，Helix 在离线和在线服务中实现了 1.86× 和 1.69× 高的解码吞吐量。 Helix 的平均延迟高于 SP，因为 SP 通过 A100 节点服务大部分请求，而 L4 和 T4 节点利用率不足。与 Swarm 相比，Helix 在离线和在线服务中实现了 1.94× 和 2.00× 高的解码吞吐量。 Helix 的提示延迟降低了 15%，但由于 Helix 的 GPU 利用率较高，解码延迟增加了 16%。与 LLaMA 30B 类似，Swarm 由于其模型放置而未充分利用 A100 节点。

我们还在模拟器中进行这些实验（参见第 6.1 节）以检查其保真度。仿真结果与实际系统评估结果一起如图 6 所示。解码吞吐量的平均误差低于 5%。提示和解码延迟的平均误差分别低于5%和4%。这表明我们的模拟器实现了高保真度，并且可以用于比较不同方法的性能。

6.4 地理分布式集群

本节使用我们在第 2 节中的高保真模拟器来评估 Helix 在地理分布式集群中在线和离线服务 LLaMA 30B 和 70B 的能力。 6.1.图 7 显示了结果。尽管此设置涉及与单个集群相同的 GPU 集，但缓慢的集群间网络会导致所有方法具有较低的吞吐量和较高的延迟。

对于 LLaMA 30B，Helix 发现的最佳模型放置仍然由分别由三种类型的机器提供服务的三个管道组成。与 SP 相比，Helix 的离线和在线服务解码吞吐量分别提高了 7% 和 10%。提示延迟降低了 14%，解码延迟提高了 2% 。与 Swarm 相比，Helix 在线和离线服务的解码吞吐量分别提高了 2.41× 和 2.33×。 Helix 还将在线服务的提示和解码延迟减少了 66% 和 24%。

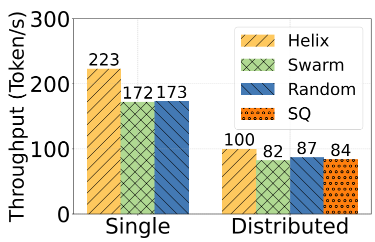
另一方面，由于更大的激活大小和模型深度，服务 LLaMA 70B 对慢速网络更敏感。 Helix 通过使用管道深度较浅的模型放置来减少网络开销。模型放置 Helix 发现，与 Swarm 相比，管道深度减少了 28%。当网络速度很快时，它也比 Helix 发现的浅 19%。 Helix 的模型放置规划器平衡网络开销与单节点 GPU 利用率，在地理分布式 GPU 集群中实现高吞吐量和低延迟。与 SP 相比，Helix 的离线和在线服务解码吞吐量分别提高了 1.61× 和 1.79×。 SP 具有较低的平均延迟，因为它使用快速 GPU 满足大多数请求，而 12 个 T4 节点由于 VRAM 限制而未得到充分利用。与 Swarm 相比，Helix 的离线和在线服务解码吞吐量分别提高了 1.92× 和 1.97×。 Helix 还将提示和解码延迟减少了 21% 和 7%。我们观察到 Swarm 离线服务期间出现严重拥塞。平均提示延迟达到71s，是Helix的7.5×。我们将在第 2 节中通过案例研究更多地讨论 Swarm 的拥塞问题。 6.7.

6.5 高 GPU 异构集群

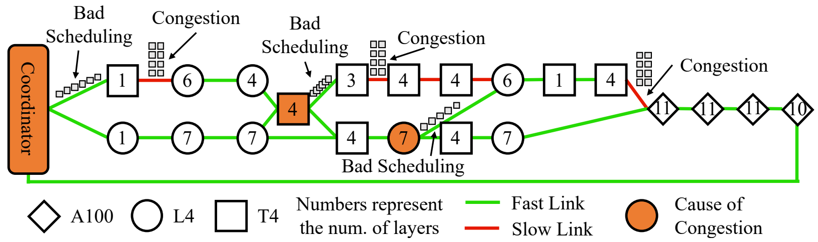
本节在具有 42 个计算节点和 7 种 GPU 的高度异构 GPU 集群上评估 Helix。图 8 显示了结果。在该集群中，V100、T4和T4×2节点无法自行形成服务管道。我们报告没有这些机器的 SP 的吞吐量。我们还尝试使用这些机器构建混合管道，并将混合管道的数量报告为 SP+。与 Swarm、SP 和 SP+ 相比，Helix 的离线服务吞吐量为 1.37×、2.91× 和 2.24×，在线服务吞吐量为 1.48×、3.29× 和 2.54×。 Helix 还分别将提示延迟减少了 17%、1% 和 7%。 Helix 的平均解码延迟稍高，因为基线未充分利用慢速 GPU，并使用具有大 VRAM 的快速 GPU 来满足大多数请求。这一结果表明，当集群中存在多种类型的 GPU 时，Helix 可以实现一致的高性能。

6.6 模型放置深入探讨

在本节中，我们分析不同模型放置方法对集群服务吞吐量的影响。我们评估了 Helix 在单个集群和地理分布式集群上的离线服务性能，并将 Helix 的模型放置方法与 Swarm 和 Petals 中使用的模型放置方法进行了比较。



(a) Decode throughput.



(b) Request scheduling case study.

Figure 10. (a) Comparing different request scheduling methods with offline serving of LLaMA 70B. **(b)** A case study that illustrates the congestion in Swarm and random scheduling when serving LLaMA 70B in geo-distributed clusters setup.

Both methods perform model placement using throughput-based heuristics. To isolate the effect of model placement, we use Helix's request scheduler for all methods. Fig. 9a shows the decode throughput. Compared with Petals and Swarm, Helix achieves 1.23× and 2.10× higher throughput on the single cluster, and 1.49× and 2.38× throughput on the geo-distributed clusters. We perform a case study on LLaMA 70B to demonstrate why Helix achieves the best performance.

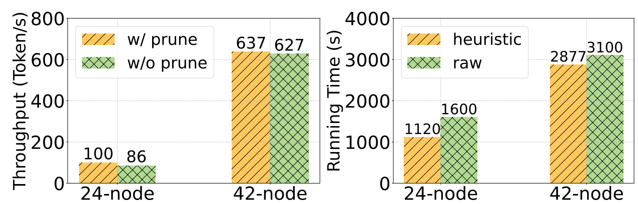
Case study: LLaMA 70B - single cluster. Fig. 9b shows the model placement and GPU compute utilization for each method when serving LLaMA 70B on the single cluster. Swarm's model placement introduces a bottleneck at the end of its pipeline, where 4 T4 nodes each serves 4 layers. This bottleneck causes GPU under-utilization on A100 and L4 nodes, significantly decreasing the serving throughput. For Petals' model placement, 8 T4 nodes and 1 L4 node are under-utilized, which negatively affects the serving throughput. For Helix's model placement, almost all nodes are fully-utilized. The efficient use of GPUs enables Helix to outperform Swarm and Petals by 2.10× and 1.23× respectively.

6.7 Request Scheduling Deep Dive

This section analyzes the impact of different request scheduling methods on the serving throughput of the cluster. We evaluate Helix's offline serving performance of LLaMA 70B on both the single cluster and geo-distributed clusters. We compare Helix's request scheduler with (1) Swarm, which schedules requests based on real-time throughput of each candidate node, and (2) random scheduling, which randomly chooses a candidate in scheduling. For the geo-distributed clusters setup, we further compare with Shortest Queue First scheduling (SQ), which always assigns requests to the node with shortest queue. To eliminate the impact of model placement, all methods use the model placement found by Helix. Fig. 10a shows the decode throughput. Compared with Swarm and random scheduling, Helix achieves 30% and 29% higher throughput on the single cluster, and 22% and 15% higher throughput on the geo-distributed clusters. Compared with Shortest Queue First scheduling, Helix achieves 19% higher decode throughput. Moreover, runtime monitoring

Table 8. Problem size with and without pruning. var means variables, and cstr means constraints.

Problem size	With pruning	Without pruning
24-node	876 var 1122 cstr	1376 var 1848 cstr
42-node	2144 var 2772 cstr	4004 var 5502 cstr



(a) Cluster pruning. **(b)** Initial values.

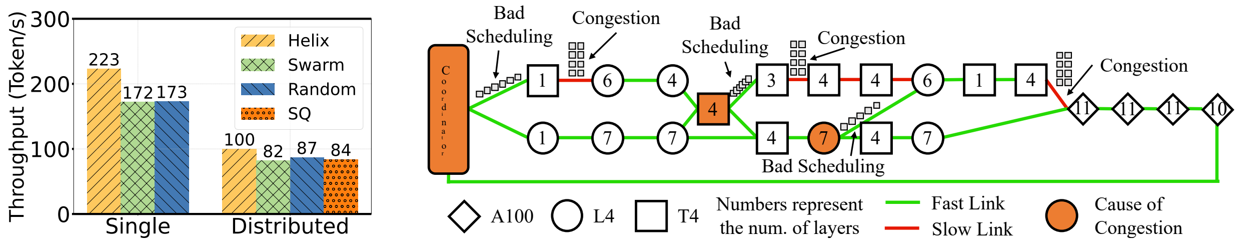
Figure 11. Ablation study on MILP optimization.

shows that all three baseline scheduling methods introduce severe congestion. We illustrate this further in the case study.

Case study: LLaMA 70B - distributed clusters. Fig. 10b shows the model placement plan found by Helix for serving LLaMA 70B on the geo-distributed clusters. The plan avoids using slow inter-cluster network connections as much as possible, but a few compute nodes are still connected with low-bandwidth connections. When using Swarm or random scheduling to schedule requests, we observe severe congestion on the three links marked as "congestion" in the figure – prompt phase requests queue up on those links for an average of 5s - 16s before they can be transmitted. We root-cause the nodes responsible for the congestion and mark them orange in the figure. Surprisingly, we find that one congestion is caused by bad scheduling from a node 3 hops away. This verifies the necessity of a global scheduling method that can take both network and compute into account. We also observe similar congestion when serving LLaMA 70B with Swarm's request scheduling on the model placement it finds for the geo-distributed clusters.

6.8 Ablation Study on Optimization

This section performs an ablation study on the two MILP optimizations introduced in Sec. 4.5. We evaluate offline serving of LLaMA 70B on the geo-distributed and highly



(a) Decode throughput. **(b)** Request scheduling case study.

图 10.(a) 将不同的请求调度方法与 LLaMA 70B 的离线服务进行比较。 (b) 案例研究 在 说明了在地理分布式集群设置中为 LLaMA 70B 提供服务时 Swarm 和随机调度中的拥塞情况。

两种方法都使用基于吞吐量的启发法来执行模型放置。为了隔离模型放置的影响，我们对所有方法使用 Helix 的请求调度程序。图 9a 显示了解码吞吐量。与 Petals 和 Swarm 相比，Helix 在单个集群上的吞吐量提高了 1.23× 和 2.10×，在地理分布式集群上的吞吐量提高了 1.49× 和 2.38×。我们对 LLaMA 70B 进行案例研究，以证明 Helix 为何能实现最佳性能。

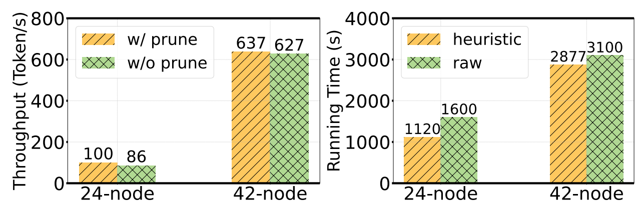
案例研究：LLaMA 70B - 单簇。图 9b 显示了在单个集群上提供 LLaMA 70B 时每种方法的模型布局和 GPU 计算利用率。 Swarm 的模型放置在其管道末端引入了瓶颈，其中 4 个 T4 节点每个服务于 4 个层。此瓶颈导致 A100 和 L4 节点上的 GPU 利用率不足，从而显着降低服务吞吐量。对于 Petals 的模型放置，8 个 T4 节点和 1 个 L4 节点未得到充分利用，这对服务吞吐量产生了负面影响。对于Helix的模型放置，几乎所有节点都被充分利用。 GPU 的高效使用使 Helix 的性能分别比 Swarm 和 Petals 高出 2.10× 和 1.23×。

6.7 请求调度深入探讨

本节分析不同请求调度方法对集群服务吞吐量的影响。我们评估了 LLaMA 70B 在单集群和地理分布式集群上的 Helix 离线服务性能。我们将Helix的请求调度器与（1）Swarm进行比较，Swarm根据每个候选节点的实时吞吐量来调度请求，以及（2）随机调度，在调度中随机选择一个候选节点。对于地理分布式集群设置，我们进一步与最短队列优先调度（SQ）进行比较，它始终将请求分配给具有最短队列的节点。为了消除模型放置的影响，所有方法都使用 Helix 找到的模型放置。图 10a 显示了解码吞吐量。与 Swarm 和随机调度相比，Helix 在单个集群上的吞吐量分别提高了 30% 和 29%，在地理分布式集群上的吞吐量分别提高了 22% 和 15%。与最短队列优先调度相比，Helix 的解码吞吐量提高了 19%。此外，运行时监控

表 8. 有和没有修剪的问题大小。 var 表示变量，cstr 表示约束。

Problem size	With pruning	Without pruning
24-node	876 var 1122 cstr	1376 var 1848 cstr
42-node	2144 var 2772 cstr	4004 var 5502 cstr



(a) Cluster pruning. **(b)** Initial values.

图 11. MILP 优化的消融研究。

表明所有三种基线调度方法都会引入严重的拥塞。我们在案例研究中进一步说明了这一点。案例研究：LLaMA 70B - 分布式集群。图 10b 显示了 Helix 发现的用于在地理分布式集群上提供 LLaMA 70B 服务的模型放置计划。该计划尽可能避免使用慢速的集群间网络连接，但少数计算节点仍然采用低带宽连接。当使用 Swarm 或随机调度来调度请求时，我们观察到图中标记为“拥塞”的三个链路上出现严重拥塞——提示阶段请求在这些链路上排队平均需要 5 秒至 16 秒才能传输。我们找出造成拥塞的节点的根本原因，并在图中将它们标记为橙色。令人惊讶的是，我们发现一次拥塞是由 3 跳之外的节点的错误调度引起的。这验证了一种能够兼顾网络和计算的全局调度方法的必要性。当使用 Swarm 的请求调度为地理分布式集群找到的模型放置提供 LLaMA 70B 服务时，我们还观察到类似的拥塞。

6.8 优化消融研究

本节对第 2 节中介绍的两种 MILP 优化进行了消融研究。 4.5.我们评估了 LLaMA 70B 在地理分布式和高度分布式上的离线服务。

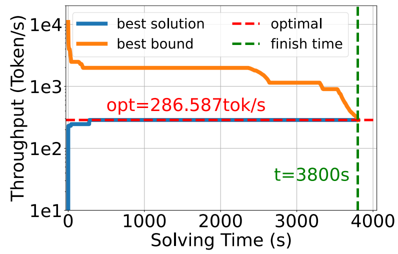


Figure 12. Best solution and best upper bound found by the MILP solver relative to solving time. The red dotted line marks the optimal throughput for this cluster.

heterogeneous clusters, referred to as the 24-node and 42-node settings respectively.

Cluster pruning. When cluster pruning is enabled, we prune network connections such that the average degree of each node is 12, which is sufficient for LLM inference systems as we discuss below. Enabling cluster pruning removes 50% and 72% network connections for 24 and 42-node settings. Table. 8 shows that pruning reduces problem size by 36% and 46% for the two settings. Fig. 11a shows that Helix achieves 16% and 2% higher decode throughput when using the model placement found with cluster pruning. We note that the amount of speed-up achieved would vary depending on the specific instance of the MILP problem at hand. Pruning slow network connections does not harm throughput because network connections used in serving is very sparse – usually each node only communicates with a few other nodes. Also, there are many equivalent model placements that can achieve the same throughput. Pruning the cluster very likely keeps some of these placements still valid. It makes the search for these placements easier with limited optimization time, as the problem size (and solution space size) is reduced.

Initial values. We compare the performance of running Helix's model placement planner starting from solutions of heuristic methods and from default values. Since the best model placements found are the same, we compare the wall clock time to find the placement. Fig. 11b shows that running MILP from heuristic solutions takes 43% and 8% less time for the 24- and 42-node setup. We note that the speed-up achieved would vary depending on the specific instance of the MILP problem at hand. The results show that starting from heuristic solutions accelerates model placement in Helix.

6.9 Model Placement Quality

This section evaluates the quality of model placements found by Helix relative to the MILP solving time. In this evaluation, we run Helix to find the optimal model placement for serving LLaMA 30B in a cluster with 4 L4 and 6 T4 machines. We record the best model placement as well as the best upper bound found by the MILP solver that we used (Gurobi) during the solving process. The best upper bound represents the best possible objective value that could be achieved for the MILP problem and will gradually become tighter as the solver explores more nodes and adds cutting planes. Fig. 12 shows the quality of the best model placement and the best upper bound relative to the solving time. Results show that Helix finds the optimal solution in less than 5 minutes, but it takes the solver more than one hour to enumerate all possible solutions and confirm the optimality. This indicates that we can early-stop the solving process, as high-quality solutions emerge in the early stages of computation.

7 Related Work

Machine Learning Model Serving There are a large number of works for serving machine learning models, discussing aspects including system implementation [36, 37], model placement [24, 41, 48, 58], request scheduling [13, 48, 63], and tail-latency mitigation [20, 21, 31]. However, due to LLM's unique auto-regressive execution paradigm, these approaches fail to efficiently serve LLMs. Instead, many recent LLM-specific systems tackle the unpredictable execution time and high memory consumption in LLM serving. Orca [61] proposed iteration level scheduling to release resources once a request is finished. vLLM [22] introduced PageAttention to further reduce the memory consumption of each request by allocating exact number of pages it requires. Speculative Inference [23, 30] applies a small model to predict multiple output tokens, and verify them in a single iteration. Splitwise [40] and DistServe [65] found that disaggregating the prompt and decode phase can improve the throughput, since the two phases have different workload characteristics. Sarathi [2] introduced chunked prefill, which allocates a budget to the prompt phase to make each microbatch's workload balanced, minimizing pipeline bubble. All above works are orthogonal to our work and can be integrated into our system, since our focus is on the cluster heterogeneity.

ML workloads on heterogeneous clusters. Several methods have utilized heterogeneous GPUs for ML tasks. Some of them [18, 39] co-design the model partition and placement on a heterogeneous cluster but assume a uniform network bandwidth. Learninghome [46] and DeDLOC [9] studied the network-aware routing on a decentralized cluster but only considers either data or pipeline parallelism individually. SWARM [45], as discussed in Sec. 2.1, optimized the pipeline communication in a heterogeneous network. However, it schedules only by the next stage's metadata, lacking a global view. There are also several efforts on using approximations to reduce network communication [57] or synchronization [16]. Most of them focus on model training. In model inference, especially LLMs, serving with heterogeneous and geo-distributed GPUs is not well studied. SkyPilot [60] and Mélange [12] select the best type of GPUs for a request, but

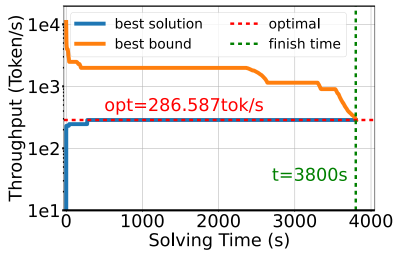


图 12. MILP 求解器找到的相对于求解时间的最佳解决方案和最佳上限。红色虚线标记了该集群的最佳吞吐量。

异构集群，分别称为 24 节点和 42 节点设置。

集群修剪。启用集群修剪后，我们会修剪网络连接，使每个节点的平均度数为 12，这对于我们下面讨论的 LLM 推理系统来说已经足够了。启用集群修剪会删除 24 和 42 节点设置的 50% 和 72% 的网络连接。桌子。图 8 显示，对于两种设置，修剪将问题大小分别减少了 36% 和 46%。图 11a 显示，当使用通过集群修剪发现的模型放置时，Helix 的解码吞吐量分别提高了 16% 和 2%。我们注意到，所实现的加速量会根据当前 MILP 问题的具体实例而有所不同。修剪慢速网络连接不会损害吞吐量，因为服务中使用的网络连接非常稀疏——通常每个节点只与少数其他节点通信。此外，还有许多等效的模型布局可以实现相同的吞吐量。修剪集群很可能会使其中一些放置仍然有效。随着问题规模（和解决方案空间大小）的减小，它使得在有限的优化时间内搜索这些位置变得更加容易。

初始值。我们从启发式方法的解决方案和默认值开始比较运行 Helix 模型放置规划器的性能。由于找到的最佳模型放置位置相同，因此我们比较挂钟时间来找到放置位置。图 11b 显示，对于 24 节点和 42 节点设置，从启发式解决方案运行 MILP 所需的时间分别减少了 43% 和 8%。我们注意到，所实现的加速会根据当前 MILP 问题的具体实例而有所不同。结果表明，从启发式解决方案开始可以加速 Helix 中的模型放置。

6.9 模型放置质量

本节评估 Helix 相对于 MILP 求解时间找到的模型布局的质量。在此评估中，我们运行 Helix 来找到在具有 4 个 L4 和 6 个 T4 机器的集群中为 LLaMA 30B 提供服务的最佳模型放置。我们记录了最佳模型放置以及我们使用的 MILP 求解器 (Gurobi) 找到的最佳上限求解过程。最佳上限代表 MILP 问题可以实现的最佳目标值，并且随着求解器探索更多节点并添加切割平面，该上限将逐渐变得更紧。图 12 显示了最佳模型放置的质量和相对于求解时间的最佳上限。结果表明，Helix 在不到 5 分钟的时间内找到了最优解，但求解器需要一个多小时才能枚举所有可能的解并确认最优性。这表明我们可以提前停止求解过程，因为高质量的解决方案会在计算的早期阶段出现。

7 相关工作

机器学习模型服务有大量的工作用于服务机器学习模型，讨论的方面包括系统实现[36,37]、模型放置[24,41,48,58]、请求调度[13,48,63]和尾部延迟缓解[20,21,31]。然而，由于LLM独特的自回归执行范式，这些方法无法有效地服务于LLM。相反，许多最近的 LLM 特定系统解决了 LLM 服务中不可预测的执行时间和高内存消耗问题。 Orca [61]提出了迭代级调度，以便在请求完成后释放资源。 vLLM [22]引入了PageAttention，通过分配其所需的确切页面数来进一步减少每个请求的内存消耗。推测推理 [23, 30] 应用一个小模型来预测多个输出标记，并在单次迭代中验证它们。 Splitwise [40] 和 DistServe [65] 发现，分解提示和解码阶段可以提高吞吐量，因为这两个阶段具有不同的工作负载特征。 Sarathi [2] 引入了分块预填充，它将预算分配给提示阶段，以使每个微批次的工作负载平衡，从而最大限度地减少管道泡沫。所有上述工作都与我们的工作正交，并且可以集成到我们的系统中，因为我们的重点是集群异构性。

异构集群上的 ML 工作负载。有几种方法利用异构 GPU 来执行 ML 任务。其中一些 [18, 39] 共同设计模型分区并放置在异构集群上，但假设网络带宽统一。 Learninghome [46] 和 DeDLOC [9] 研究了去中心化集群上的网络感知路由，但仅单独考虑数据或管道并行性。 SWARM [45]，如第 2 节中所讨论的。 2.1、优化了异构网络中的管道通信。然而，它仅根据下一阶段的元数据进行调度，缺乏全局视图。还有一些努力使用近似来减少网络通信[57]或同步[16]。他们大多数都专注于模型训练。在模型推理中，尤其是大语言模型构和地理分布式 GPU 的服务尚未得到很好的研究。 SkyPilot [60] 和 Mélange [12] 为请求选择最佳的 GPU 类型，但是

each request is served by a single GPU type. Petals [5], as discussed in Sec. 2.1, studies a decentralized pipeline parallel setup. It designs a greedy model allocation and request scheduling for a dynamical device group, losing optimizing opportunities for a fixed device group. HexGen [19] is a concurrent work on LLM serving in heterogeneous clusters. It is based on fixed pipelines (see Sec. 5.1) and adopts heuristic-based methods to search for the optimal model placement. In comparison, our Max Flow formulation and per-request pipeline is more flexible than HexGen and our MILP formulation can guarantee the optimal solution.

Scheduling for Heterogeneous Resources There exists extensive research on scheduling algorithms for heterogeneous resources. For example, energy-aware scheduling [25] in the Linux kernel schedules tasks for heterogeneous CPU topologies. There are also several works [4, 43, 54] that focus on general scheduling in heterogeneous clusters. Due to the auto-regressive nature of LLM inference, these methods cannot be directly used for heterogeneous LLM serving.

8 Conclusion

This paper presents Helix, the first high-throughput, low-latency LLM serving engine for heterogeneous GPU clusters, with guaranteed optimal solution that maximizes throughput. Helix formulates and solves the model placement and request scheduling as a Max-Flow problem. Compared to existing solutions, Helix achieves significant improvements in throughput and latency.

Acknowledgment

We thank the anonymous reviewers and our shepherd Íñigo Goiri for their valuable feedback and constructive suggestions that helped improve this paper. We also express our gratitude to the Google Cloud Innovator program for providing the machines on Google Compute Engine for our experiments. This work was supported in part by a Sloan Foundation Fellowship and a VMware Systems Research Award. This work was also partially supported by the National Science Foundation under grant numbers CNS-2147909, CNS-2211882, and CNS-2239351, along with gift awards from Amazon and Meta.

A Artifact

A.1 Abstract

Our artifacts include Helix's simulator and prototype system for distributed LLM serving in heterogeneous and geo-distributed clusters. The code implements key algorithms including the MaxFlow-based LLM serving formulation, MILP-based model placement planner, and per-request pipeline scheduler. We also provide comprehensive documentation and scripts for environment setup and system execution from scratch. The artifacts contain the identical simulator and prototype system used in Helix's evaluation in the paper. The simulator can run on a single machine, while the prototype system requires a cluster deployment.

A.2 Artifact check-list (meta-information)

- **Algorithm:** MaxFlow-based LLM serving formulation; MILP-based model placement planner; per-request pipeline scheduler
- **Compilation:** The simulator is implemented in Python. The prototype system's inter-node communication framework is written in C++ (recommended to compile with GCC 13.2) and includes an automated compilation script with CMake, while its remaining components are implemented in Python.
- **Model:** We use LLaMa-2 70B as the test workload. The prototype system operates with dummy weights, requiring only the model architecture specification (which is provided in our code repository).
- **Data set:** We use the Azure Conversation Dataset as our evaluation traces, with a pre-parsed version included in our code repository.
- **Run-time environment:** For the simulator, we recommend using Python 3.10, while it can also support other recent Python versions. The simulator is not sensitive to OS versions. For the prototype system, we recommend using Ubuntu 24.04 LTS to setup the environment. We also recommend using conda to isolate the run-time environment for both systems. For detailed software dependencies, please refer to Sec. A.3.3.
- **Hardware:** The simulator runs on a single machine without specific hardware requirements. The prototype system runs on a cluster of 24 machines with NVIDIA GPUs. Please refer to Sec. A.3.2 for detailed hardware requirements.
- **Metrics:** We evaluate using the same metrics as presented in the paper: decoding throughput, decoding latency, and prompt processing latency.
- **Output:** Results are logged to both terminal output and files. We include expected results in the code repository.
- **How much disk space required (approximately)?:** The total size of all log files is around 300 MB.
- **How much time is needed to prepare workflow (approximately)?:** The environment setup takes around 1 - 2 hours.
- **How much time is needed to complete experiments (approximately)?:** Functionality evaluation takes around 2 hours. Reproducibility evaluation takes around 16 hours. (The part that needs the whole 24 machine cluster is around 4 hours. **Namely, the sections that need the whole cluster are part of Sec 6.3, 6.6 and 6.7.**)
- **Publicly available?:** Yes. DOI: 10.5281/zenodo.14037926.
- **Code licenses (if publicly available)?:** Apache 2.0

A.3 Description

A.3.1 How to access. The artifact is publicly available at https://github.com/Thesys-lab/Helix-ASPLOS25. It is also archived as 10.5281/zenodo.14037926. The file size is around 300 MB. Please refer to our Github repository for the latest version.

A.3.2 Hardware dependencies. The simulator runs on a single machine without specific hardware requirements. We recommend at least 32 GB of memory for simulating large

每个请求均由单一 GPU 类型提供服务。花瓣 [5]，如第 2 节所述。 2.1，研究分散式管道并行设置。它为动态设备组设计了贪婪模型分配和请求调度，失去了固定设备组的优化机会。 HexGen [19] 是一项关于在异构集群中服务的 LLM 的并发工作。它基于固定管道（参见第 5.1 节），并采用启发式方法来搜索最佳模型放置。相比之下，我们的 Max Flow 公式和按请求管道比 HexGen 更灵活，并且我们的 MILP 公式可以保证最佳解决方案。

异构资源的调度 对于异构资源的调度算法存在着广泛的研究。例如，Linux 内核中的能量感知调度 [25] 为异构 CPU 拓扑调度任务。还有一些工作[4,43,54]关注异构集群中的通用调度。由于LLM推理的自回归性质，这些方法不能直接用于异构LLM服务。

8 结论

本文介绍了 Helix，这是第一个用于异构 GPU 集群的高吞吐量、低延迟 LLM 服务引擎，具有保证吞吐量最大化的最佳解决方案。 Helix 将模型放置和请求调度作为最大流问题来制定和解决。与现有解决方案相比，Helix 在吞吐量和延迟方面实现了显着改进。

致谢

我们感谢匿名审稿人和我们的牧羊人 Íñigo Goiri 提供的宝贵反馈和建设性建议，帮助改进了本文。我们还对 Google Cloud Innovator 计划表示感谢，感谢他们为我们的实验提供了 Google Compute Engine 上的机器。这项工作得到了斯隆基金会奖学金和 VMware 系统研究奖的部分支持。这项工作还得到了国家科学基金会的部分支持，拨款号为 CNS-2147909、CNS-2211882 和 CNS-2239351，以及亚马逊和 Meta 的礼品奖。

神器

A.1 摘要

我们的产品包括 Helix 的模拟器和原型系统，用于在异构和地理分布式集群中提供分布式 LLM 服务。该代码实现了关键算法，包括基于 MaxFlow 的 LLM 服务公式、基于 MILP 的模型放置规划器和每个请求的管道调度器。我们还提供全面的文档和脚本，用于从头开始进行环境设置和系统执行。这些工件包含相同的模拟器以及论文中 Helix 评估中使用的原型系统。模拟器可以在单机上运行，而原型系统需要集群部署。

A.2 工件检查表（元信息）

- 算法：基于MaxFlow的LLM服务制定；基于MILP的模型布局规划器；每个请求的管道调度程序
- 编译：模拟器是用Python实现的。原型系统的节点间通信框架是用C++ (编写的，建议使用GCC 13.2)编译，并包含一个使用CMake的自动编译脚本，而其其余组件则用Python实现。
- 型号：我们使用LLaMa-2 70B作为测试工作负载。原型系统使用虚拟权重运行，仅需要模型架构规范（在我们的代码存储库中提供）。
- 数据集：我们使用 Azure 对话数据集作为我们的评估跟踪，并在我们的代码存储库中包含预解析的版本。
- 运行环境：对于模拟器，我们推荐使用Python 3.10，同时它也可以支持其他最新的Python版本。模拟器对操作系统版本不敏感。对于原型系统，我们建议使用 Ubuntu 24.04 LTS 来搭建环境。我们还建议使用 conda 来隔离两个系统的运行时环境。详细的软件依赖关系请参阅第 2 节。 A.3.3。
- 硬件：模拟器在单机上运行，没有特定的硬件要求。该原型系统在 24 台配备 NVIDIA GPU 的机器组成的集群上运行。请参阅第 2 节。 A.3.2 了解详细的硬件要求。
- 指标：我们使用与论文中提供的相同指标进行评估：解码吞吐量、解码延迟和提示处理延迟。
- 输出：结果记录到终端输出和文件中。我们将预期结果包含在代码存储库中。
- 需要多少磁盘空间（大约）？ ：所有日志文件的总大小约为 300 MB。
- 准备工作流程需要多少时间（大约）？ ：环境设置大约需要 1 - 2 小时。
- 完成实验需要多长时间（大约）？ ：功能评估大约需要 2 小时。再现性评估大约需要 16 小时。 （需要整个24机集群的部分大约需要4个小时。即需要整个集群的部分是第6.3、6.6和6.7节的部分。）
- 公开吗？ ：是的。 DOI：10.5281/zenodo.14037926。
- 代码许可证（如果公开可用）？ ：Apache 2.0

A.3 描述

A.3.1 如何访问。该工件可在 https://github.com/Thesys-lab/Helix-ASPLOS25 上公开获取。它还存档为 10.5281/zenodo.14037926。文件大小约为 300 MB。请参阅我们的 Github 存储库以获取最新版本。

A.3.2 硬件依赖性。该模拟器在单机上运行，没有特定的硬件要求。我们建议至少 32 GB 内存来模拟大型

clusters to prevent out-of-memory issues. The prototype system requires cluster deployment - our example configuration uses 24 machines, consisting of 4 machines with 1×A100-40GB, 8 machines with 1×L4, and 12 machines with 1×T4. We recommend to set up each machine with at least 16 CPU cores and 128 GB memory to avoid out-of-memory issues. The network connection between machines should ideally be at least 10 Gbps, with latency of approximately a few milliseconds. Usually machines in the same region from common cloud providers can meet the network requirements. For functionality test purposes, it is also possible to use machines from different regions.

A.3.3 Software dependencies. We recommend running the simulator with Python 3.10. It relies on networkx, matplotlib and gurobipy. The MILP-based model placement planner uses Gurobi as the MILP solver. Running the example in the code base does not require additional licenses. However, if you want to run model placement for larger clusters, it is necessary to acquire a Gurobi license that does not limit problem size. We recommend running the prototype system on Ubuntu 24.04 LTS and with Python 3.10. To build the inter-node communication framework, you need to install build-essential, cmake, libzmq, cppzmp and pybind11. To run the prototype system, you need to install CUDA 12.6 and vLLM 0.4.0.post1. For a step-by-step guide to setting up the environment and running the experiments, please refer to https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md.

A.3.4 Data sets. We use the Azure Conversation Dataset as our evaluation traces, with a pre-parsed version included in our code repository.

A.3.5 Models. We use LLaMa-2 70B as the test workload. The prototype system operates with dummy weights, requiring only the model architecture specification.

A.4 Detailed Steps for Reproducing Results

We provide an example to show the functionality of our system, please refer to https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md. We also provide another example to reproduce all result we got in the paper, please refer to https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/artifact_evaluation/ae_readme.md

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[4] Rashmi Bajaj and Dharma P Agrawal. Improving scheduling of tasks in a heterogeneous environment. *IEEE Transactions on Parallel and Distributed Systems*, 15(2):107–118, 2004.

[5] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. *arXiv preprint arXiv:2209.01188*, 2022.

[6] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin A Raffel. Distributed inference and fine-tuning of large language models over the internet. *Advances in Neural Information Processing Systems*, 36, 2024.

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[8] Joseph Cheriyan and SN Maheshwari. Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6):1057–1086, 1989.

[9] Michael Diskin, Alexey Bukhtiyarov, Max Ryabinin, Lucile Saulnier, Anton Sinitsin, Dmitry Popov, Dmitry V Pyrkin, Maxim Kashirin, Alexander Borzunov, Albert Villanova del Moral, et al. Distributed deep learning in open collaborations. *Advances in Neural Information Processing Systems*, 34:7879–7897, 2021.

[10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[11] Google Cloud. Google cloud compute products. https://cloud.google.com/products/compute, 2024. Accessed: 2024-03-13.

[12] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. M\'elange: Cost efficient large language model serving by exploiting gpu heterogeneity. *arXiv preprint arXiv:2404.14527*, 2024.

[13] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462, 2020.

[14] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023.

[15] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[16] Kevin Hsieh, Aaron Harlap, Nandita Vijaykumar, Dimitris Konomis, Gregory R Ganger, Phillip B Gibbons, and Onur Mutlu. Gaia:{Geo-Distributed} machine learning approaching {LAN} speeds. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 629–647, 2017.

[17] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[18] Xianyan Jia, Le Jiang, Ang Wang, Wencong Xiao, Ziji Shi, Jie Zhang, Xinyuan Li, Langshi Chen, Yong Li, Zhen Zheng, et al. Whale: Efficient giant model training over heterogeneous {GPUs}. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 673–688, 2022.

[19] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. Hexgen: Generative inference of large language model over

集群以防止内存不足问题。原型系统需要集群部署 - 我们的示例配置使用 24 台机器，其中包括 4 台具有 1×A100-40GB 的机器、8 台具有 1×L4 的机器和 12 台具有 1×T4 的机器。我们建议将每台计算机设置为至少 16 个 CPU 核心和 128 GB 内存，以避免内存不足问题。理想情况下，机器之间的网络连接应至少为 10 Gbps，延迟约为几毫秒。通常常见云提供商的同一区域的机器就可以满足网络要求。出于功能测试的目的，也可以使用来自不同地区的机器。

A.3.3 软件依赖性。我们建议使用 Python 3.10 运行模拟器。它依赖于networkx、mat-plotlib和gurobipy。基于 MILP 的模型布局规划器使用 Gurobi 作为 MILP 求解器。在代码库中运行示例不需要额外的许可证。但是，如果您想为更大的集群运行模型放置，则需要获取不限制问题大小的 Gurobi 许可证。我们建议在 Ubuntu 24.04 LTS 和 Python 3.10 上运行原型系统。构建节点间通信框架需要安装build-essential、cmake、libzmq、cppzmp和pybind11。要运行原型系统，您需要安装CUDA 12.6和vLLM 0.4.0.post1。有关设置环境和运行实验的分步指南，请参阅 https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md。

A.3.4 数据集。我们使用 Azure 对话数据集作为我们的评估跟踪，并在我们的代码存储库中包含预解析的版本。

A.3.5 模型。我们使用 LLaMa-2 70B 作为测试工作负载。原型系统使用虚拟权重运行，仅需要模型架构规范。

A.4 再现结果的详细步骤

我们提供了一个示例来展示我们系统的功能，请参阅https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md。我们还提供了另一个示例来重现我们在论文中得到的所有结果，请参阅https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/artifact_evaluation/ae_readme.md

参考

[1] Josh Achiam、Steven Adler、Sandhini Agarwal、Lama Ahmad、Ilge Akkaya、Florencia Leoni Aleman、Diogo Almeida、Janko Altenschmidt、Sam Altman、Shyamal Anadkat 等。 Gpt-4 技术报告。 arXiv 预印本 arXiv：2303.08774，2023。 [2] Amey Agrawal、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav S Gulavani 和 Ramachandran Ramjee。 Sarathi：通过使用分块预填充进行解码，实现高效的 llm 推理。 arXiv 预印本 arXiv:2308.16369, 2023。

[3] Joshua Ainslie、James Lee-Thorp、Michiel de Jong、Yury Zemlyanskiy、Federico Lebrón 和 Sumit Sanghai。 Gqa：从多头检查点训练广义多查询变压器模型。 arXiv 预印本 arXiv:2305.13245, 2023。 [4] Rashmi Bajaj 和 Dharma P Agrawal。改进异构环境中的任务调度。 IEEE Transactions on Parallel and Distributed Systems, 15(2):107–118, 2004。 [5] Alexander Borzunov、Dmitry Baranchuk、Tim Dettmers、Max Ryabinin、Younes Belkada、Artem Chumachenko、Pavel Samygin 和 Colin Raffel。 Petals：大型模型的协作推理和微调。 arXiv 预印本 arXiv：2209.01188，2022。 [6] Alexander Borzunov、Max Ryabinin、Artem Chumachenko、Dmitry Baranchuk、Tim Dettmers、Younes Belkada、Pavel Samygin 和 Colin A Raffel。通过互联网对大型语言模型进行分布式推理和微调。神经信息处理系统进展，36，2024。 [7] Tom Brown、Benjamin Mann、Nick Ryder、Melanie Subbiah、Jared D Kaplan、Prafulla Dhariwal、Arvind Neelakantan、Pranav Shyam、Girish Sastry、Amanda Askell 等。语言模型是小样本学习者。神经信息处理系统的进展，33：1877-1901，2020。 [8] Joseph Cheriyan 和 SN Maheshwari。最大网络流量的预流推送算法分析。 SIAM 计算杂志，18(6)：1057–1086，1989。 [9] Michael Diskin、Alexey Bukhtiyarov、Max Ryabinin、Lucile Saulnier、Anton Sinitsin、Dmitry Popov、Dmitry V Pyrkin、Maxim Kashirin、Alexander Borzunov、Albert Villanova del Moral 等。开放协作中的分布式深度学习。神经信息处理系统进展，34：7879–7897，2021。 [10] Abhimanyu Dubey，Abhinav Jauhri，Abhinav Pandey，Abhishek Kadian，Ahmad Al-Dahle，Aiesha Letman，Akhil Mathur，Alan Schelten，Amy Yang，Angela Fan，等。骆驼 3 群模型。 arXiv 预印本 arXiv:2407.21783, 2024。 [11] 谷歌云。谷歌云计算产品。 https://cloud.google。 com/products/compute，2024 年。访问时间：2024 年 3 月 13 日。 [12] Tyler Griggs、Xiaoxuan Liu、Jiayang Yu、Doyoung Kim、Wei-Lin Chiang、Alvin Cheung 和 Ion Stoica。 M\'elange：通过利用 GPU 异构性提供具有成本效益的大型语言模型。 arXiv 预印本 arXiv:2404.14527, 2024。 [13] Arpan Gujarati、Reza Karimi、Safya Alzayat、Wei hao、Antoine Kaufmann、Ymir Vigfusson 和 Jonathan Mace。像发条一样服务 {DNN}：自下而上的性能可预测性。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20)，第 443–462 页，2020 年。 [14] Gurobi Optimization, LLC。 Gurobi优化器参考手册，2023。 [15] Sirui Hong，Xiawu Cheng，Jonathan Chen，Yuheng Cheng，Jinlin Wang，Ceyao Zhang，Zili Wang，Steven Ka Shing Yau，Zijuan Lin，Liyang Zhou，et al。 Metagpt：多智能体协作框架的元编程。 arXiv 预印本 arXiv：2308.00352，2023。 [16] Kevin Hsieh、Aaron Harlap、Nandita Vijaykumar、Dimitris Konomis、Gregory R Ganger、Phillip B Gibbons 和 Onur Mutlu。 Gaia：{地理分布式}机器学习接近{LAN}速度。第 14 届 USENIX 网络系统设计与实现研讨会 (NSDI 17)，第 629-647 页，2017 年。 [17] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe：使用管道并行性对巨型神经网络进行高效训练。神经信息处理系统进展, 32, 2019. [18] 贾贤彦, 蒋乐, 王昂, 肖文聪, 石子吉, 张杰, 李新元, 陈朗石, 李勇, 郑振, 等. Whale：通过异构 {GPU} 进行高效的巨型模型训练。 2022 年 USENIX 年度技术会议（USENIX ATC 22），第 673-688 页，2022 年。 [19] Youhe Jiang，Ran Yan，Xiaozhe Yao，Yang Zhou，Beidi Chen 和 Binhang Yuan。 Hexgen：大型语言模型的生成推理

heterogeneous environment. In *Forty-first International Conference on Machine Learning*, 2024.

[20] Jack Kosaian, KV Rashmi, and Shivaram Venkataraman. Parity models: erasure-coded resilience for prediction serving systems. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 30–46, 2019.

[21] Jack Kosaian, KV Rashmi, and Shivaram Venkataraman. Learning-based coded computation. *IEEE Journal on Selected Areas in Information Theory*, 1(1):227–236, 2020.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[23] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[24] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, 2023.

[25] Linux Kernel Documentation. Energy aware scheduling. https://docs.kernel.org/scheduler/sched-energy.html, 2024. Linux Kernel Documentation.

[26] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

[27] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. Realhf: Optimized rlhf training for large language models through parameter reallocation, 2024.

[28] Meta. Introducing Meta Llama 3: The most capable openly available LLM to date — ai.meta.com. https://ai.meta.com/blog/meta-llama-3/. [Accessed 07-05-2024].

[29] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023.

[30] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024.

[31] Hema Venkata Krishna Giri Narra, Zhifeng Lin, Ganesh Ananthanarayanan, Salman Avestimehr, and Murali Annavaram. Collage inference: Using coded redundancy for lowering latency variation in distributed image classification systems. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, pages 453–463. IEEE, 2020.

[32] NVIDIA. NVIDIA A100 Tensor Core GPU. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf. Accessed: 2024-10-28.

[33] NVIDIA. NVIDIA H100 Tensor Core GPU. https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet. Accessed: 2024-10-28.

[34] NVIDIA. NVIDIA L4 Tensor Core GPU. https://resources.nvidia.com/en-us-data-center-overview/l4-gpu-datasheet. Accessed: 2024-10-28.

[35] NVIDIA. NVIDIA T4 Tensor Core GPU. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf. Accessed: 2024-10-28.

[36] NVIDIA. Triton Inference Server.

[37] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139*, 2017.

[38] OpenAI. ChatGPT, 2023.

[39] Jay H Park, Gyeongchan Yun, M Yi Chang, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh, and Young-ri Choi. {HetPipe}: Enabling large {DNN} training on (whimpy) heterogeneous {GPU} clusters through integration of pipelined model parallelism and data parallelism. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 307–321, 2020.

[40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[41] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.

[42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[43] Andrei Radulescu and Arjan JC Van Gemund. Fast and effective task scheduling in heterogeneous systems. In *Proceedings 9th heterogeneous computing workshop (HCW 2000)(Cat. No. PR00556)*, pages 229–238. IEEE, 2000.

[44] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[45] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. Swarm parallelism: Training large models can be surprisingly communication-efficient. In *International Conference on Machine Learning*, pages 29416–29440. PMLR, 2023.

[46] Max Ryabinin and Anton Gusev. Towards crowdsourced training of large neural networks using decentralized mixture-of-experts. *Advances in Neural Information Processing Systems*, 33:3659–3672, 2020.

[47] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[48] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. Nexus: A gpu cluster engine for accelerating dnn-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 322–337, 2019.

[49] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[50] Foteini Strati, Paul Elvinger, Tolga Kerimoglu, and Ana Klimovic. Ml training with cloud gpu shortages: Is cross-region the answer? In *Proceedings of the 4th Workshop on Machine Learning and Systems*, pages 107–116, 2024.

[51] Seyed Mohammadhossein Tabatabaee, Jean-Yves Le Boudec, and Marc Boyer. Interleaved weighted round-robin: A network calculus analysis. *IEICE Transactions on Communications*, 104(12):1479–1493, 2021.

[52] The ZeroMQ authors. ZeroMQ An open-source universal messaging library, 2024.

[53] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 497–513, Boston, MA, April 2023. USENIX Association.

异构环境。第四十一届机器学习国际会议，2024 年。 [20] Jack Kosaian、KV Rashmi 和 Shivaram Venkataraman。奇偶校验模型：预测服务系统的纠删码弹性。第 27 届 ACM 操作系统原理研讨会论文集，第 30-46 页，2019 年。 [21] Jack Kosaian、KV Rashmi 和 Shivaram Venkataraman。基于学习的编码计算。 IEEE Journal on Selected Areas in Information Theory, 1(1):227−236, 2020. [22] Woosuk Kwon、Zhuohan Li、Siyuan Zhuang、Ying Shen、Lianmin Cheng、Codyhao Yu、Joseph Gonzalez、Hao Zhu 和 Ion Stoica。通过 pagedattention 服务的大型语言模型的高效内存管理。收录于 2023 年 ACM SIGOPS 第 29 届操作系统原理研讨会论文集。 [23] Yaniv Leviathan、Matan Kalman 和 Yossi Matias。通过推测解码从 Transformer 进行快速推理。国际机器学习会议，第 19274-19286 页。 PMLR，2023。 [24] 李卓瀚，郑联民，钟银民，刘文森，盛英，金鑫，黄艳平，陈志峰，张浩，Joseph E Gonzalez，等。 {AlpaServe}：具有模型并行性的统计多路复用，用于深度学习服务。第 17 届 USENIX 操作系统设计与实现研讨会 (OSDI 23)，第 663−679 页，2023 年。[25] Linux 内核文档。能源意识调度。 https://docs.kernel.org/scheduler/sched-energy.html，2024。Linux 内核文档。 [26] 罗紫阳，徐灿，赵璞，孙庆丰，耿秀波，胡文祥，陶重阳，马静，林庆伟，蒋大新。 Wizard- coder：使用 evol-instruct 为大型语言模型提供支持。 arXiv 预印本 arXiv:2306.08568, 2023。 [27] 梅志宇、付伟、李凯伟、王光举、张焕辰和吴一。 Realhf：通过参数重新分配优化大型语言模型的 rlhf 训练，2024 年。[28] Meta。 Meta Llama 3 简介：迄今为止最有能力的公开大语言模型eta.com。 https://ai.meta.com/blog/meta-llama-3/。 [访问时间：2024 年 7 月 5 日]。 [29] 苗旭鹏，Gabriele Oliaro，张志浩，程新浩，金宏毅，陈天琪，贾志浩。迈向高效的生成式大语言模型服务：从算法到系统的调查。 arXiv 预印本 arXiv:2312.15234, 2023。 [30] 苗旭鹏，Gabriele Oliaro，张志豪，程新浩，王泽宇，张正新，Rae Ying Yee Wong，Alan Zhu，Lijie Yang，Xiaoxiang Shi，等。 Specinfer：通过基于树的推测推理和验证来加速大型语言模型。第 29 届 ACM 国际编程语言和操作系统架构支持会议记录，第 3 卷，第 932-949 页，2024 年。 [31] Hema Venkata Krishna Giri Narra、Zhifeng Lin、Ganesh Ananthanarayanan、Salman Avestimehr 和 Murali Annavaram。拼贴推理：使用编码冗余来降低分布式图像分类系统中的延迟变化。 2020 年 IEEE 第 40 届分布式计算系统国际会议 (ICDCS)，第 453-463 页。 IEEE，2020。[32] NVIDIA。 NVIDIA A100 张量核心 GPU。 https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf。访问时间：2024 年 10 月 28 日。 [33]英伟达。 NVIDIA H100 张量核心 GPU。 https://resources.nvidia。 com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet。访问时间：2024 年 10 月 28 日。 [34]英伟达。 NVIDIA L4 张量核心 GPU。 https://resources.nvidia.com/en-us-data-center-overview/l4-gpu-datasheet。访问时间：2024 年 10 月 28 日。 [35]英伟达。 NVIDIA T4 张量核心 GPU。 https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf。访问时间：2024 年 10 月 28 日。 [36]英伟达。 Triton 推理服务器。

[37] Christopher Olston、Noah Fiedel、Kiril Gorovoy、Jeremiah Harmsen、Li Lao、Fangwei Li、Vinu Rajashekhar、Sukriti Ramesh 和 Jordan Soyke。 Tensorflow-serving：灵活、高性能的机器学习服务。 arXiv 预印本 arXiv:1712.06139, 2017。 [38] OpenAI。 ChatGPT，2023。[39]Jay H Park、Gyeongchan Yun、M Yi Chang、Nguyen T Nguyen、Seungmin Lee、Jaesik Choi、Sam H Noh 和 Young-ri Choi。 {HetPipe}：通过集成流水线模型并行性和数据并行性，在（奇特的）异构 {GPU} 集群上启用大型 {DNN} 训练。 2020 年 USENIX 年度技术会议 (USENIX ATC 20)，第 307-321 页，2020。 [40] Pratyush Patel、Esha Choukse、Chaojie Zhang、Aashaka Shah、Íñigo Goiri、Saeed Maleki 和 Ricardo Bianchini。 Splitwise：使用相分裂的高效生成 llm 推理。 2024 年 ACM/IEEE 第 51 届计算机架构国际研讨会 (ISCA)，第 118-132 页。IEEE，2024。[41] Reiner Pope、Sholto Douglas、Aakanksha Chowdhery、Jacob Devlin、James Bradbury、Jonathan Heek、Kefan Xiao、Shivani Agrawal 和 Jeff Dean。有效扩展变压器推理。机器学习和系统论文集，5，2023 年。 [42] Alec Radford、Jeffrey Wu、Rewon Child、David Luan、Dario Amodei、Ilya Sutskever 等人。语言模型是无监督的多任务学习者。 OpenAI 博客，1(8):9，2019。 [43] Andrei Radulescu 和 Arjan JC Van Gemund。异构系统中快速有效的任务调度。第九届异构计算研讨会论文集 (HCW 2000)（目录号 PR00556），第 229-238 页。 IEEE，2000。 [44] Baptiste Roziere、Jonas Gehring、Fabian Gloeckle、Sten Sootla、Itai Gat、Xiaoqing Ellen Tan、Yossi Adi、Jingyu Liu、Tal Remez、Jérémy Rapin 等。 Code llama：开放代码基础模型。 arXiv 预印本 arXiv：2308.12950，2023。 [45] Max Ryabinin、Tim Dettmers、Michael Diskin 和 Alexander Borzunov。集群并行性：训练大型模型的通信效率令人惊讶。国际机器学习会议，第 29416-29440 页。 PMLR，2023。[46]马克斯·里亚宾宁和安东·古谢夫。使用分散的专家组合对大型神经网络进行众包训练。神经信息处理系统的进展，33：3659−3672，2020。[47]诺姆·沙泽尔。快速变压器解码：您只需要一个写入头。 arXiv 预印本 arXiv:1911.02150, 2019。 [48] Haichen Shen、Lequn Chen、Yuchen Jin、Liangyu Zhu、Bingyu Kong、Matthai Philipose、Arvind Krishnamurthy 和 Ravi Sundaram。 Nexus：一个 GPU 集群引擎，用于加速基于 DNN 的视频分析。第 27 届 ACM 操作系统原理研讨会论文集，第 322-337 页，2019 年。 [49] Mohammad Shoeybi、Mostofa Patwary、Raul Puri、Patrick LeGresley、Jared Casper 和 Bryan Catanzaro。 Megatron-lm：使用模型并行性训练数十亿参数语言模型。 arXiv 预印本 arXiv：1909.08053，2019。 [50] Foteini Strati、Paul Elvinger、Tolga Kerimoglu 和 Ana Klimovic。云 GPU 短缺的机器学习训练：跨区域是答案吗？第四届机器学习和系统研讨会论文集，第 107-116 页，2024 年。 [51] Seyed Mohammadhossein Tabatabaee、Jean-Yves Le Boudec 和 Marc Boyer。交错加权循环：网络演算分析。 IEICE Transactions on Communications，104(12):1479−1493, 2021。 [52] ZeroMQ 作者。 ZeroMQ 开源通用消息库，2024 年。[53] John Thorpe、Pengzhan Zhao、Jonathan Eyolfson、Yifan Qiao、Zhihao Jia、Minjia Zhang、Ravi Netravali 和Guoqing Harry Xu。 Bamboo：使抢占式实例具有弹性，可以负担得起大型 DNN 的训练。第 20 届 USENIX 网络系统设计和实现研讨会 (NSDI 23)，第 497-513 页，马萨诸塞州波士顿，2023 年 4 月。USENIX 协会。

[54] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE transactions on parallel and distributed systems*, 13(3):260–274, 2002.

[55] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[56] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[57] Jue Wang, Yucheng Lu, Binhang Yuan, Beidi Chen, Percy Liang, Christopher De Sa, Christopher Re, and Ce Zhang. Cocktailsgd: Fine-tuning foundation models over 500mbps networks. In *International Conference on Machine Learning*, pages 36058–36076. PMLR, 2023.

[58] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. Transparent {GPU} sharing in container clouds for deep learning workloads. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 69–85, 2023.

[59] x.ai. Annocing grok.

[60] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, et al. {SkyPilot}: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 437–455, 2023.

[61] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[62] Binhang Yuan, Yongjun He, Jared Davis, Tianyi Zhang, Tri Dao, Beidi Chen, Percy S Liang, Christopher Re, and Ce Zhang. Decentralized training of foundation models in heterogeneous environments. *Advances in Neural Information Processing Systems*, 35:25464–25477, 2022.

[63] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. {MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1049–1062, 2019.

[64] Shiwei Zhang, Lansong Diao, Chuan Wu, Zongyan Cao, Siyu Wang, and Wei Lin. Hap: Spmd dnn training on heterogeneous gpu clusters with automated program synthesis. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 524–541, 2024.

[65] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[54] Haluk Topcuoglu、Salim Hariri 和 Min-You Wu。用于异构计算的高性能且低复杂度的任务调度。并行和分布式系统上的 IEEE 事务，13(3):260–274, 2002。 [55] Hugo Touvron、Thibaut Lavril、Gautier Izacard、Xavier Martinet、Marie-Anne Lachaux、Timothée Lacroix、Baptiste Rozière、Naman Goyal、Eric Hambro、Faisal Azhar 等。 Llama：开放高效的基础语言模型。 arXiv 预印本 arXiv:2302.13971, 2023。 [56] Hugo Touvron、Louis Martin、Kevin Stone、Peter Albert、Amjad Almahairi、Yasmine Babaei、Nikolay Bashlykov、Soumya Batra、Prajjwal Bhargava、Shruti Bhosale 等。 Llama 2：开放基础和微调的聊天模型。 arXiv 预印本 arXiv:2307.09288, 2023。 [57] 王珏、陆玉成、袁斌航、陈贝迪、梁波西、克里斯托弗·德萨、克里斯托弗·雷和张策。 Cocktailsgd：在 500mbps 网络上微调基础模型。国际机器学习会议，第 36058-36076 页。 PMLR，2023。 [58]吴冰洋，张自立，白志豪，刘轩哲，金鑫。容器云中透明的 {GPU} 共享用于深度学习工作负载。第 20 届 USENIX 网络系统设计和实现研讨会 (NSDI 23)，第 69-85 页，2023 年。 [59] x.ai。宣布格鲁克。 [60] 杨宗衡，吴章豪，迈克尔·罗，蒋伟林，罗米尔·巴德瓦吉，Woosuk Kwon，庄思源，弗兰克·斯飞·栾，高塔姆·米塔尔，斯科特·申克，等。 {SkyPilot}：天空的云间代理

计算。第 20 届 USENIX 网络系统设计与实现研讨会 (NSDI 23)，第 437–455 页，2023 年。 [61] Gyeong-In Yu、Joo Seong Jeong、Geon-Woo Kim、Soojeong Kim 和 Byung-Gon Chun。 Orca：用于{基于 Transformer} 生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)，第 521-538 页，2022 年。 [62] Binhang Yuan、Yongjun He、Jared Davis、Tianyi 张、Tri Dao、Beidi Chen、Percy S Liang、Christopher Re 和 Ce Zhu。异构环境中基础模型的分散训练。神经信息处理系统进展，35：25464–25477，2022。[63]Chengliang Zhang，Minchen Yu，Wei Wang，and Feng Yan。 {MArk}：利用云服务实现{经济有效的}、{SLO 感知}机器学习推理服务。 2019年USENIX年度技术会议（USENIX ATC 19），第1049-1062页，2019。 [64] Shiwei Zhang，Lansong Diao，Chuan Wu，Zongyan Cao，Siyu Wang 和Wei Lin。 Hap：通过自动程序合成对异构 GPU 集群进行 Spmd DNN 训练。第十九届欧洲计算机系统会议论文集，第 524-541 页，2024 年。 [65] Yinmin Zhu、Shengyu Liu、Junda Chen、Jianbo Hu、Yibo Zhu、Xuanzhe Liu、Xin Jin 和 Hao Zhang。 {DistServe}：分解预填充和解码，以实现良好的优化大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)，第 193-210 页，2024 年。